CORVINUS UNIVERSITY of BUDAPEST

Machine Learning in Practice I

Antal Jakovac, 2025

About the course

- Course Title: "Machine Learning in Practice I"
- instructor: antal.jakovac@uni-corvinus.hu
- course objectives
  - why and when to use Machine Learning (ML)
  - data handling methods
  - core ML models before deep learning
  - how to build an ML project

About the course

- about the course
  - evaluation: written exams (together with exercises)
  - learning material: course slides, books, podcasts, etc.
  - books:
    - An Introduction to Statistical Learning
      (G.James , D. Witten , T. Hastie , R. Tibshirani, https://www.statlearning.com/)
    - Designing machine learning systems (Chip Huyen)
  - github repository: https://github.com/ajakovac/ML-in-Practice-I

About the course

- about the course
  - evaluation: written exams (together with exercises)
  - learning material: course slides, books, podcasts, etc.
  - books:
    - An Introduction to Statistical Learning
      (G.James , D. Witten , T. Hastie , R. Tibshirani, https://www.statlearning.com/)
    - Design
  - github rep

```
#git clone project from command line
git clone https://github.com/ajakovac/ML-in-Practice-I.git
cd ML-in-Practice-I
#read the README.md file and follow instructions
```

Introduction

Introduction

- why do we need Artificial Intelligence (Machine Learning)?

	observation	modelling	computation	conclude/action
historic times	human	human	human	human
machines	**machine**	human	human	human
computers	**machine**	human	**machine**	human
AI	**machine**	**machine**	**machine**	human

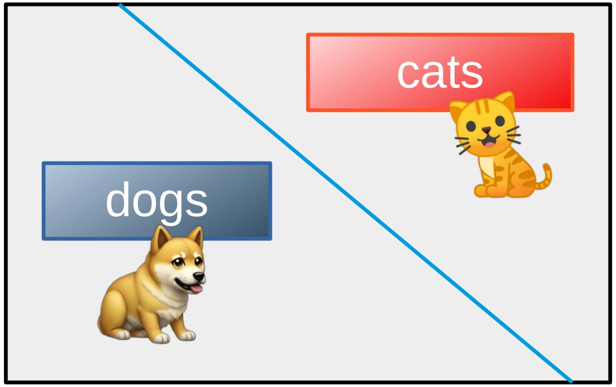
- task of AI is not to solve an actual problem, but solve modelling
- we do not know the model it uses → it can be smarter than us!

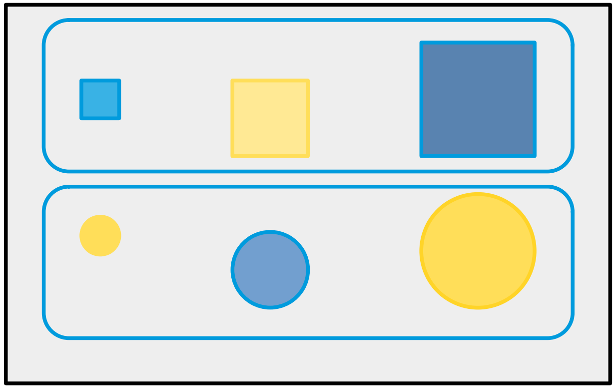
Introduction

- Traditional Programming vs. ML
  - traditional: data + rules → output
  - ML: data + output → rules
- when to use ML?
  - Automation of complex tasks → heuristics
  - Adaptability
  - Performance on large data
  - Real-world examples (email spam, face detection, recommendation systems)

Introduction

- Types of ML; traditionally
  - supervised:
    - show data + labels → learns the assignment
    - examples: classification, regression
  - unsupervised:
    - find patterns in unlabeled data
    - not unique → context, magnitude
    - examples: clustering, dimensionality reduction
  - reinforcement:
    - learn by interacting with the environment
    - not covered in this course





Introduction

- Types of ML; other points of view
  - task-driven (System I)
    - define a task (classification, prediction, control)
    - show examples of good (eventually bad) solutions
    - train models to optimize for success on that task
    - includes most supervised/reinforcement learning examples
  - data-driven (System II)
    - reveal structures by showing similar data
    - does not need explicit labelling
    - LLM’s, autoencoders, similarity learning
    - recently popular (self-supervised learning, foundation models, and retrieval-based systems)

Introduction

- Typical ML tasks
  - classification
    - face, dog breeds, bird songs classification
    - spam detection, disease diagnosis, sentiment analysis
    - lot of labeled data
  - regression (price prediction, weather forecasting, energy demand)
  - clustering (customer segmentation, document grouping, biological cell types)
  - compression (PCA, visualization, autoencoders, embeddings)

Introduction

- Typical ML tasks
  - decision making (robotics, game playing, resource allocation, monitoring)
  - generation (text, translation, image, data, molecular structure, code)
  - outlier analysis (email spam, suspicious customer, fraud detection, etc.)
  - recommendation systems (movies, jobs, code completion, etc.)
  - ranking/retrieval (search engines, document/data retrieval)

Introduction

- somewhat overlapping areas; e.g. chemical plant monitoring system
  - anomaly detection
  - classification
  - time-series forecasting
  - reinforcement learning (control), recommendation systems

Introduction

- ML algorithms (pre-deep neural networks)
  - Linear regression, logistic probability
  - Bayesian analysis, probability theory
  - Support Vector Machines
  - Decision Trees, ensemble methods
  - k-nearest neighbour method
  - Principal Component Analysis
  - Extreme Learning Machine
- majority of ML applications in production are not DNN!

Introduction

Tools

- language: Python
- packages:
  - numpy: numerical computing
  - scipy, scikit-learn: classical ML tools
  - pandas: data manipulation
  - matplotlib, seaborn: visualization

Introduction

Tools

- Python virtual environment:
```
> python -m venv .venv
```
- activate environment:
  - Linux/Windows WSL:
  ```
  > source .venv/bin/activate
  ```
  - Windows PowerShell:
  ```
  > .venv\Scripts\Activate
  ```
- Install necessary packages within the environment
```
> pip install matplotlib numpy scipy scikit-learn pandas seaborn ipykernel
```
- automation:
```
> pip freeze > requirements.txt
> pip install -r requirements.txt
```

Introduction

Tools for code development

- notebook services: Jupyter, Google Colab
- advanced editors: Visual Studio Code or similar
- Linux-like environment in Windows: WSL
- docker: containerize your work – like a standalone machine
- communication: API endpoints (Python fastAPI or Flask)

Typical ML workflow

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

- what is the problem we want to solve? (classification, regression, clustering, …)
- is ML an adequate tool?
- what does success look like? (accuracy? low error? business impact?)
- what are the domain constraints and goals (e.g. medicine data availability, privacy → practical, ethical or technical limits)
- example: predict customer churn or classify handwritten digits

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

> - enough data
> - sampling methods → diverse data
> - labelled data (if supervised)
> - example: downloading a CSV of housing prices or collecting user logs

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

- missing values, duplicates, outliers
- normalize, scale, encode categorical features
- convert raw inputs into usable formats
- example: fill missing ages with median; scale prices to [0,1]

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

- create meaningful input features
- transform or combine raw attributes
- select or construct new features based on domain knowledge
- example: from date of birth → compute age; combine “city” and “job type”

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

- choose a suitable algorithm (e.g. linear regression; SVM; RF; k-NN)
- consider interpretability, complexity, training time
- example: try logistic regression first for classification, then maybe a random forest, or a deep neural network

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

- split data into training and validation sets (and possibly test)
- train on one part, validate on another
- tune hyperparameters (e.g., via cross-validation)
- example: train a model on 80% of the data, validate on 20%

Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

Use metrics appropriate for the task:

- classification: accuracy, precision, recall, F1
- regression: MSE, MAE, R^2
- plot confusion matrices, ROC curves, residuals
- example: check precision/recall on spam classification

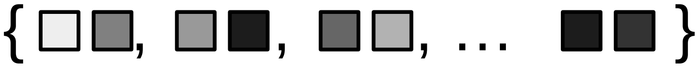
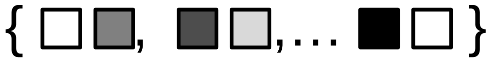
Introduction

- Typical ML workflow
  1) Problem definition
  2) Data collection
  3) Data cleaning & preprocessing
  4) Feature engineering
  5) Model selection
  6) Training & validation
  7) Evaluation
  8) Deployment & monitoring

- integrate the model into production (web app, API, etc.)
- monitor for drift, performance degradation
- retrain if needed (based on new data or feedback)
- example: host model on a server and track prediction accuracy over time

Logics of Intelligence

Logics of Intelligence

- Consider a world W consisting of 2 pixels
  
- W also has two visible states A and B. We observe that
  - A → 
  - B → 
- **Step1**: problem definition: tell the state from the pixels
  → classification task

Logics of Intelligence

- **Step2-3**: Data collection, cleaning, preprocessing
- in our case: observation → numerics
- the pixels are black-and-white → brightness is a number in [0,1]
  - black: 0
  - white: 1
  - grey: somewhere in between
- 2 pixels mean 2 numbers (x_1, x_2)
- *usually Step2-3 is rather tedious…*

Logics of Intelligence

- **Step4-5-6**: feature engineering, model selection, training (in practice these are separate steps)
- Let us plot the point pairs corresponding to state A and B
  - the points lie in subspaces (lines)
  - within state A and B

$$\frac{x_1}{2}+x_2=\begin{cases}1 & \text{for state } A\\ 0 & \text{for state } B\end{cases}$$

  - worth to introduce new coordinates (features)

$$u=\frac{x_1}{2}+x_2,\quad v=x_1-\frac{x_2}{2}$$

Logics of Intelligence

- **Step4-5-6**: feature engineering, model selection, training
- with the new coordinates $x_1 = \frac{2u}{5} + \frac{4v}{5}, \; x_2 = \frac{4u}{5} - \frac{2v}{5}$
- constant u – aligned with states; constant v – changes within a given state

Logics of Intelligence

- **Step4-5-6**: feature engineering, model selection, training
- To tell apart state A and B we need only the value of u!
  - u = 1.0   state A
  - u = 0.5   state B
  - otherwise neither
- solves classification task

Logics of Intelligence

- **Step4-5-6**: feature engineering, model selection, training
- To describe a instance belonging to a state we need just v!
  - state A: v in [-0.5,0.75]
  - state B: v in [-0.25,1]
- solves compression,
  dimensional reduction

Logics of Intelligence

- **Step4-5-6**: feature engineering, model selection, training
- a perfect **feature selection** provides a **coordinate system** best aligned with the equivalence classes of the data
  - there are coordinates that are constant on each data class, and have different values on different classes (*relevant/selective coordinates* – good for classification)
  - there are coordinates that change within a given data class (*descriptive/irrelevant coordinates* – good for compression)
- **task of all model building is to find (approximately) these coordinates/features**

Logics of Intelligence

- **Step4-5-6**: feature engineering, model selection, training
- in practice we separate the task of coordination
  - **feature selection**: original features, simple combinations
  - **model selection**: single out a parametrizable functional space to combine the features (distance, linear- or nonlinear combinations)
  - **training**: determine the free parameters of the model that fits the data the best

Logics of Intelligence

- **Step4-5-6**: feature engineering, model selection, training
- **mind vs data?**
  - data/models importance ratio → data are central importance in real world applications
  - are data enough? do we possess the necessary knowledge?
  - present approach: data engineering is the most important, few fundamental research (cf. LeCun vs Wang in Meta)

Logics of Intelligence

- **Step7:** evaluation
- try to estimate how well the original task was solved
  - class reconstruction accuracy
  - robustness, proclivity for failures
  - treatment of outliers
  - business benefits

Logics of Intelligence

- **Step8:** deployment and monitoring
- in a real-life application the ML code is used as a product
- requires continuous monitoring
  - data/environment may change
  - new classes appear
  - errors may occur

Logics of Intelligence

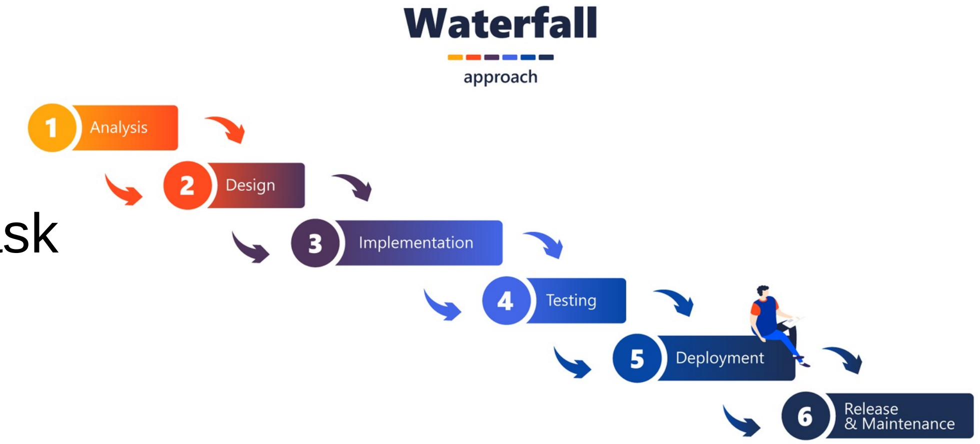
takehome message: Task of the intelligence is to

find the **relevant features** and **adapt** their values to the task

Organizing ML projects

Organizing ML projects

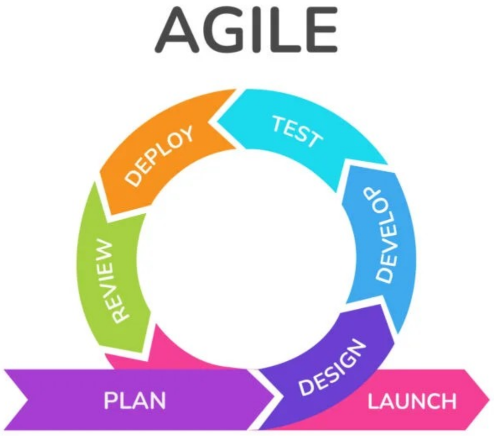
- two popular workflow schemes today: **waterfall** vs. **agile**
- **waterfall** workflow:



  - clear documentation of the task
  - predictable timeline
  - inflexible
  - late discovery of errors
- **good for well-defined projects** (like bridge building)

Organizing ML projects

- **agile** workflow: (agile manifesto, description)
  - individuals > processes
    - collaborative vs rigid teams
  - working model > detailed documentation
    - Minimal Viable Product MVP
    - improvement



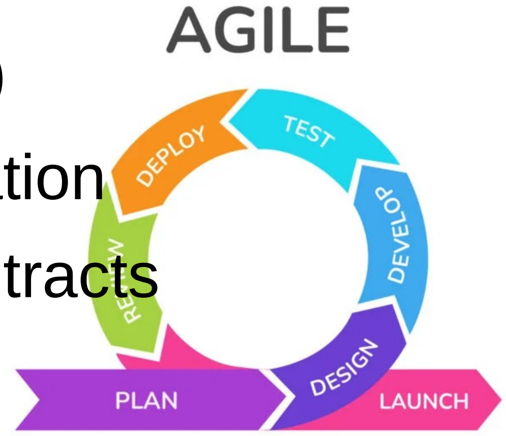
“If you’re not embarrassed by the first version of your product, you’ve launched too late.”
— Reid Hoffman, founder of LinkedIn

Goal: **Solve transportation**

- MVP: Skateboard
- Next: Bicycle
- Then: Motorcycle
- Finally: Car

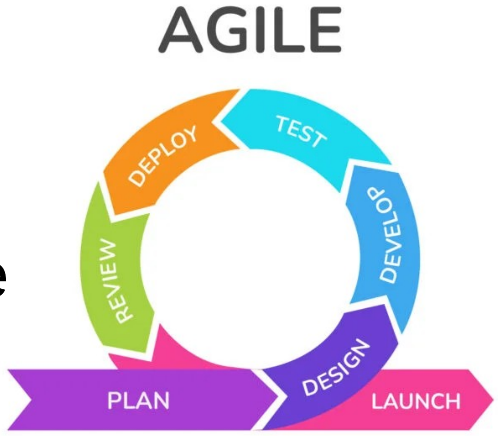
Organizing ML projects

- **agile** workflow: (agile manifesto, description)
  - customer collaboration > contract negotiation
    - customer needs even over signed contracts
  - responding to change > following a plan
    - original ideas/original task estimates may fail
    - continuous adaptation
  - launch a “good enough” product



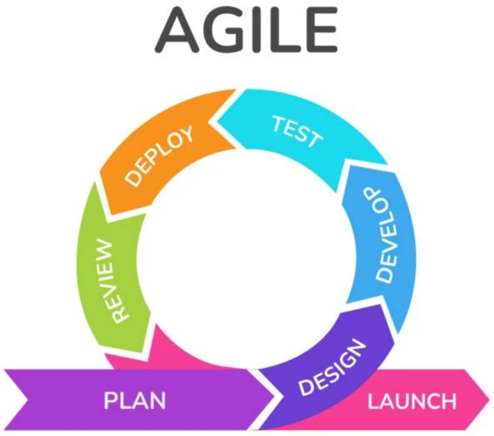
Organizing ML projects

- **agile** workflow: (agile manifesto, description)
  - pros:
    - flexible, self-improving, communicative
    - customer-centric
  - cons:
    - over-emphasized meetings (“agile theater”)
    - over-controlled
    - less predictable



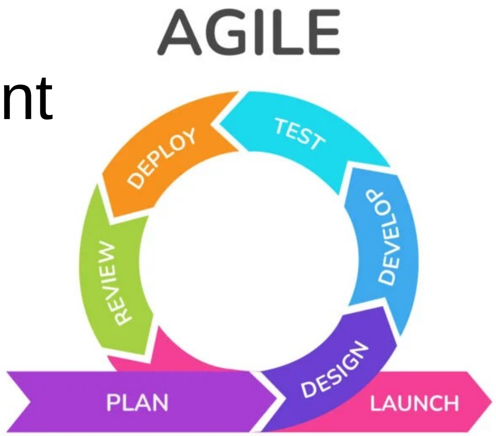
Organizing ML projects

- **agile** workflow: (agile manifesto, description)
  - loops, sprints
  - user interface/user experience (UI/UX)
  - CI/CD, CI/CD pipelines
    continuous integration, continuous deployment
  - Kanban/Jira: project development visualization
    (backlog, to do, in progress, testing, done)



Organizing ML projects

- typical small group agile project
  - define the goal (user stories), create a backlog (list of tasks)
  - make repo (github), branches
  - make MVP, backend API, frontend, testbed
  - get MVP working, deploy to test environment
  - collect feedback (team + users)
  - refine tasks, and directions to proceed



Organizing ML projects

Structure of a professional ML organization

- 🌐 Business side
  - client/stakeholder: business requirements (e.g., “reduce equipment downtime,” “recommend better products”)
  - Business Analyst / Product Manager (translates business goals into ML problems, acts as the bridge between client and technical teams)
  - Project/ Product Manager (budgeting, timelines, and resource allocation, Coordinates with business analysts and technical leads to estimate cost)

Organizing ML projects

Production level ML organization

- 🧠 Data/model experts
  - data engineer: build and maintain data pipelines → data collection, cleaning, validating, transformation (ETL/ELT), storing (database, data lake), monitoring
  - ML engineer/data scientist
    - feature generation
    - design, train, and fine-tune ML models
    - explores and experiments with data, build prototypes
    - performance evaluation

Organizing ML projects

Production level ML organization

- 🏗️ Software & Infrastructure
  - software architect: designs overall system, service planning, integration with other services, APIs, databases
  - software developer (coder): implements algorithms, frontend-backend
  - DevOps/MLOps engineer:
    - automates deployment, monitoring, scaling of ML models
    - works with Docker, Kubernetes, CI/CD pipelines, cloud platforms.
    - ensures the ML system is reproducible, reliable, and easy to maintain.

Organizing ML projects

Production level ML organization

- 🛡️ Operations, Monitoring & Safety
  - QA / Test Engineer: tests software and ML pipelines (unit tests, integration tests, stress tests), ensures reliability before release
  - Monitoring & Reliability Engineer: monitors system health after deployment.; tracks data drift, performance degradation, unusual patterns
  - Security Engineer: ensures compliance with data privacy, protects against adversarial attacks, model leaks.

Organizing ML projects

🎭 The Full Pipeline in a Story (user story)

- Client says: “I need early warnings for dangers in a chemical plant.”
- Business Analyst refines: “This means anomaly detection on sensor signals.”
- Data Engineer sets up pipelines to collect sensor data.
- Data Scientist explores, builds a prototype anomaly detector.
- ML Engineer optimizes the model for latency and accuracy.

Organizing ML projects

🎭 The Full Pipeline in a Story (user story)

- Software Architect decides how it fits into the production system.
- Developers implement the service around the model.
- DevOps/MLOps deploys it to the cloud with monitoring.
- QA & Monitoring check performance and flag issues.
- Security Engineer ensures compliance and robustness.

Programming code structure

Code structure

How to organize a (ML) code?

- points of view:
  - effective code development (also in teams)
  - scalability
  - extensibility
  - easy maintenance

monolith code structure

Original approach: monolith program structure

- UI, business logic, data access in one code
- coding logic*: input data → transformation → resulting data*
- best fits for imperative/procedural languages (like C, Fortran)
  - program state: mutable variables
  - program is a chain of commands, changing the state
- natural approach → original programming style

monolith code structure

```
y = 3 #global state variable

def function(x):
    x +=y
    y+=1 #changes y tacitly
    return x
```

problems:

- hard to trace bugs or change code:
  - variable change appear in different functions
  - frequently hidden state change in procedures
  - in a large code intractable structure
- functions are specific → not reusable
- experience: monolith codes become unmaintainable *(software complexity crisis in the 1970s–80s)*
- still used for starting a project (MVP)

programming paradigms

different type of solutions:

- object oriented programming
  - data and the corresponding procedures are bundled: ”objects” of the world
  - abstraction:
    hierarchy of classes
  - inheritance: classes are reusable
  - examples: C++, Java

```
class MyClass:
    def __init__(self, state):
        self.state = state

    #change of state is a class method
    def rotate(self, angle):
        self.state → self.state’(angle)
        return self
```

programming paradigms

- functional programming
  - stateless programs, unmutable variables
  - functions give back new objects
    (pure functions)
  - based on mathematical logic (λ-calculus)
    e.g. λx. x+1 and their application, e.g. (λx. x+1) 5 → 6
  - basic elemental functions: see next page

```
counter=0
def impure():
    counter+=1 #side effect
    return counter
def pure(x):
    return x+1 #no side effect
```

programming paradigms

- basic elemental functions
  - map: map (λx. x*2) [1,2,3] → [2,4,6]
  - reduce/fold: fold (+) 0 [1,2,3] → 6
  - filter: filter even [1,2,3,4] → [2,4]
  - zip: zip [‘a’,’b’,’c’] [1,2,3] → [(‘a’,1),(‘b’,2),(‘c’,3)]
  - compose: (compose f g) x → f(g(x))
- examples: Haskell, Lisp, F#
- not popular in the original form, but lot of elements are applied

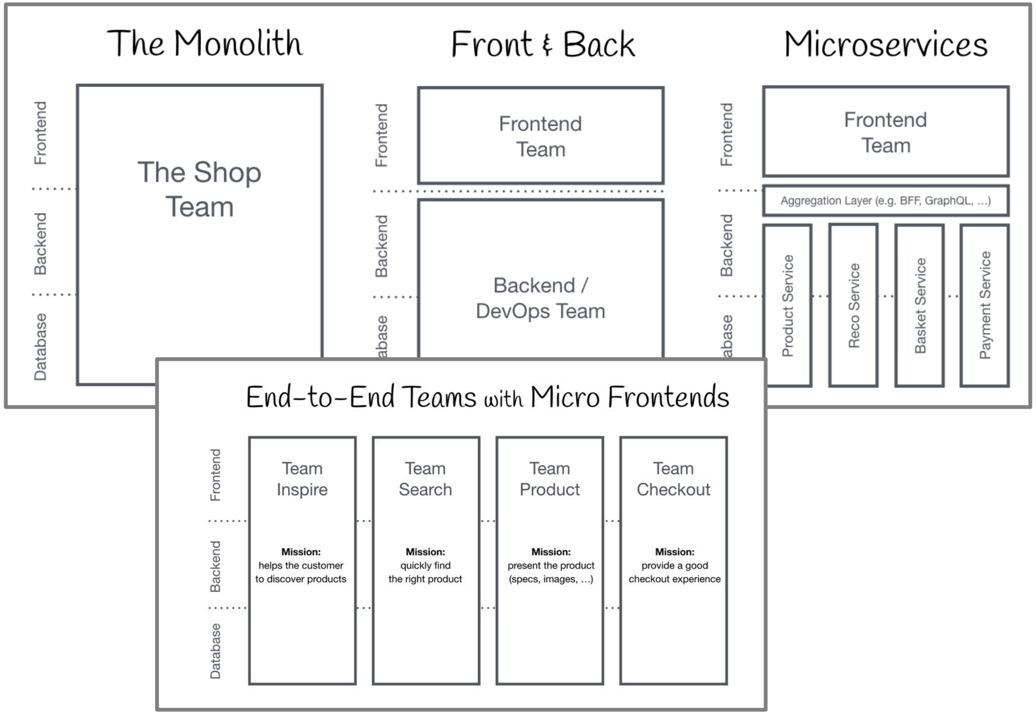
programming paradigms

- mixed (multi-paradigm) languages:
  - allow all kind of programming style
  - examples: Python, JavaScript, Rust, Swift,
  - e.g. Python has map, filter, reduce, based on lists.
  - Python supports OOP, classes, inheritance, etc.

architectures

Other way of deviating from monolithic imperative coding

- monolithic structure
- frontend+backend
- microservices
- micro frontend structure
  (c.f. https://micro-frontends.org/)



backend + frontend

Backend + frontend

- rise of web (1990s) browsers became universal clients
- specialized tools for browser programming
  (like HTML, CSS, JavaScript)
- task solving (backend) + visualization (frontend) separately
  - develop separately, different languages, tools
  - scale BE and FE separately
  - needs interface management → see next page

backend + frontend

- functions designed for intra-code communication, can not be applied any more
- inter-service communication standards:
  - APIs (application programming interface → synchronous)
  - queues (message buffer → asynchronous)

backend + frontend

- APIs – call server functions (endpoints)
  - needs continuously running background processes (web server process), listening to input channels (ports)
  - endpoints: well defined input and output formats, usually in json
  - communication → REST API: get, put/patch, post, delete
  - in Python: Flask, fastAPI
  - can be asynchronous, too

```
#fastAPI snippet
from fastapi import FastAPI
app = FastAPI()
@app.get("/")
def read_root():
    return {"message": "Hello ✨"}
----------------------------
#start service
uvicorn main:app
----------------------------
#call service
open http://127.0.0.1:8000, or
curl http://127.0.0.1:8000/
results {"message": "Hello ✨"}
```

backend + frontend

- queues: async send message
  - message broker runs in background, it stores messages
  - worker processes deliver messages for subscribers/consumers
  - data plain text or json
  - metadata + payload
  - in Python: queue + threading

```
#queue application example
import queue
q = queue.Queue()

#put data to queue
q.put(item)

#read data from queue
item = q.get()
#... process data ...
q.task_done()
```

microservices

Microservices + frontend

- good scaling properties (e.g. web users) → requiring independence of elementary tasks became popular
- lot independent, narrowly scoped (micro) services
  - separate development (languages, tools, teams), deployment
  - separate scaling
  - resilience: bugs are confined, not affect the whole system
- examples Netflix, Amazon → hype in 2010s

microservices

- new challenge: how to organize a lot of services
- microservice management: docker, Kubernetes

```
#example Dockerfile
FROM python:3.12-slim
WORKDIR /app
COPY main.py .
RUN pip install fastapi uvicorn
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "8000"]
```

```
#example docker-compose.yml
services:
  service1:
    build: ./service1
    container_name: service1
    ports:
      - "8001:8000"

  service2:
    build: ./service2
    container_name: service2
    ports:
      - "8002:8000"
----------------------------------------------
docker-compose up --build
```

microservices

- new challenge: how to organize a lot of services
- CI/CD pipelines:
  - continuous improvement
  - continuous deployment
  - team communication
- example: github

microservices

- new challenge: how to organize a lot of services
- CI/CD pipelines:
  - continuous improvement
  - continuous deployment
  - team communication
- example: github

```yaml
#example .github/workflows/ci.yml
name: CI
on:
  push: branches: ["main"]
  pull_request: branches: ["main"]
jobs:
  build-and-test:
    runs-on: ubuntu-latest
    steps:
      - name: Checkout code
        uses: actions/checkout@v4
      - name: Set up Python
        uses: actions/setup-python@v5
        with:
          python-version: "3.12"
      - name: Install dependencies
        run: |
          python -m pip install --upgrade pip
          pip install -r requirements.txt
          pip install pytest
```

microservices

- new challenge: how to organize a lot of services
- CI/CD pipelines:
  - continuous imp
  - continuous deployment
  - team communication
- example: github

```
#practical example: git clone project

git clone https://github.com/ajakovac/ML-in-Practice-I.git
cd ML-in-Practice-I
```

microservices

- despite all advantages microservices have drawbacks
  - operational complexity → manage hundreds of services
  - network calls → latency, performance issues
  - interprocess debugging is hard

microservices

- Current trends (2020s → now)
  - modular monolith applications
  - microservices where they make sense
  - functions-as-a-service (FaaS)
    - in cloud services like AWS, Azure, Google cloud
    - locally they seem like functions
    - behave like services when deployed
    - run on demand

microservices

- Current trend
  - modular m
  - microserv
  - functions-
    - in clou
    - locally
    - behave
    - run on

```
#example in AWS
#lambda_function.py
def lambda_handler(event, context):
        name = event.get("name", "my_name")
        return { "statusCode": 200,
                     "body": f"Hello, {name}! ✨"}
----------------------------------------
zip function.zip lambda_function.py
aws lambda create-function \
  --function-name helloLambda \
  --runtime python3.12 \
  --role arn:aws:iam::<YOUR_ACCOUNT_ID>:role/lambda-ex-role \
  --handler lambda_function.lambda_handler \
  --zip-file fileb://function.zip
-----------------------------------------------
curl -X POST \
    -H "Content-Type: application/json" \
    -d '{"name": "MLcourse"}' \
    https://abcd1234.execute-api.us-east-1.amazonaws.com/default/hello
```

ML Code structure

Code infrastructure → software architecture

- use few necessary microservices → clear task definition
- use lambda for simple services
- containerize microservices → portability, environmental stability
- use container management → docker (Kubernetes)
- use CI/CD management → github or similar

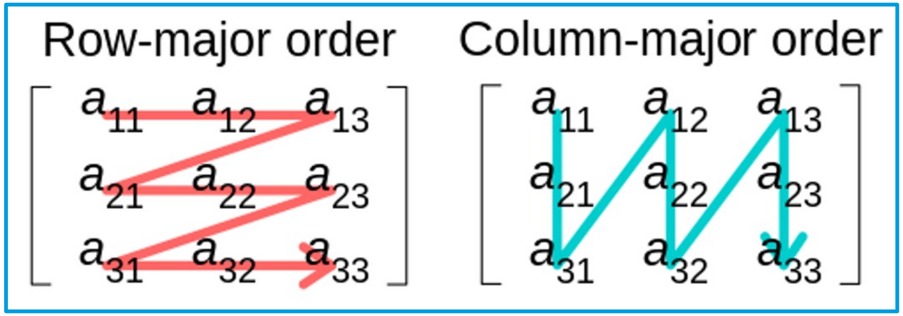
Data management

Data management

data sources:

- user input (text, image, video, etc.) – often not well formed, errors
- system generated (logs, measurements, predictions) – well formed data, easy-to-use
- own data, own clients (first-party), other companies, their own clients (second-party), other companies not own clients (third-party – e.g. internet usage, traffic, etc)

Data management



data formats

- CSV (comma separated values)
  - text, human readable
  - tabular, row-major
  - fast to get data belonging to the same example
- Parquette
  - binary – more compact, but not human readable
  - tabular, column major
  - fast to get data belonging to the same feature

Data management

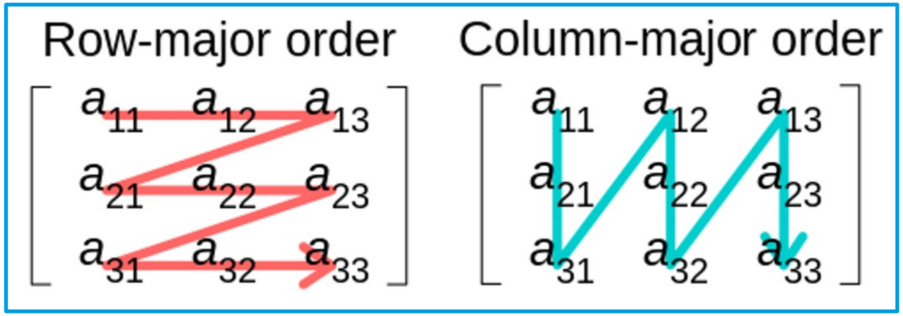
data formats

- CSV (comma
  - text, human
  - tabular, row
  - fast to get c
- Parquette
  - binary – mo
  - tabular, colu
  - fast to get c

```
#example books.csv
title,author,publisher,year
The Hobbit,J.R.R. Tolkien,George Allen & Unwin,1937
The Fellowship of the Ring,J.R.R. Tolkien,George Allen & Unwin,1954
Nineteen Eighty-Four,George Orwell,Secker & Warburg,1949
Brave New World,Aldous Huxley,Chatto & Windus,1932
The Catcher in the Rye,J.D. Salinger,"Little, Brown and Company",1951
----------------------------------------
#read data using pandas
import pandas as pd
df = pd.read_csv("books.csv")
print(df)
----------------------------------------
#output
                        title            author                  publisher  year
0                  The Hobbit    J.R.R. Tolkien       George Allen & Unwin  1937
1  The Fellowship of the Ring    J.R.R. Tolkien       George Allen & Unwin  1954
2        Nineteen Eighty-Four     George Orwell           Secker & Warburg  1949
3             Brave New World     Aldous Huxley            Chatto & Windus  1932
4      The Catcher in the Rye     J.D. Salinger  Little, Brown and Company  1951
```

er

Data management



data formats

- CSV (comma separated values)
  - text, human readable
  - tabular, row-major
  - fast to get data belonging to the same example
- Parquette
  - binary – mor
  - tabular, colu
  - fast to get da

```
#store in parquette – needs fastparquet to be installed
df.to_parquet("books.parquet", engine="fastparquet", index=False)
--------------------------
#store in parquette – needs fastparquet to be installed
df_parquet = pd.read_parquet("books.parquet", engine="fastparquet")
```

Data management

data formats:

- JSON (JavaScript Object Notation) → today's standard
  - hierarchical structure
  - text, human readable
  - key-value pairs
  - in Python: dict
  - supported by all languages

Data management

data formats:

- JSON (JavaS
  - hierarchical
  - text, human
  - key-value p
  - in Python: c
  - supported b

```
#example books.json
[
  {
    "title": "The Hobbit",
    "author": "J.R.R. Tolkien",
    "publisher": "George Allen & Unwin",
    "year": 1937
  },
  {
    "title": "The Fellowship of the Ring",
    "author": "J.R.R. Tolkien",
    "publisher": "George Allen & Unwin",
    "year": 1954
  },...
]
--------------------------------------------
#read with pandas
df = pd.read_json("books.json")
```

Data management

data models:

- for more complicated data, and if queries are required, storing data in simple files is not enough
- instead: databases with dedicated database handlers
  - relational (e.g. Sqlite)
  - nonrelational (e.g. Redis, Mongo)

Data management

data models:

- relational
  - tabular logic: main tables, subtables for standardization, index tables for faster query
  - strict schemas
  - query language for data retrieval → SQL (structured query language): declarative, specify the result, not the algorithm
  - **pros**: simple logic, widely used
  - **cons**: abundant schema systems, lots of superficial data, tedious to introduce new features, complicated table structure

Data management

data models:

- relational
  - tabular logi… …r faster query…
  - strict schem…
  - query langu… declarative,…
  - **pros**: simpl…
  - **cons**: abun… introduce n…

```
#example sqlite application – built in Python package
import sqlite3
#create/connect database
conn = sqlite3.connect("Data/books.db")
cur = conn.cursor()
#create tables
cur.execute("""
CREATE TABLE IF NOT EXISTS publishers (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    name TEXT NOT NULL
)
""")
cur.execute("""
CREATE TABLE IF NOT EXISTS books (
    id INTEGER PRIMARY KEY AUTOINCREMENT,
    title TEXT NOT NULL,
    author TEXT NOT NULL,
    year INTEGER,
    publisher_id INTEGER,
    FOREIGN KEY (publisher_id) REFERENCES publishers(id)
)
""")
```

Data management

data models:

- relational
  - tabular logi… …r faster query…
  - strict schem…
  - query langu… declarative,…
  - **pros**: simpl…
  - **cons**: abun… introduce n…

```
#insert a publisher
cur.execute("INSERT INTO publishers (name) VALUES (?)", ("George Allen &
Unwin",))
publisher_id = cur.lastrowid
# Insert a book linked to that publisher
cur.execute("INSERT INTO books (title, author, year, publisher_id) VALUES
(?, ?, ?, ?)",("The Hobbit", "J.R.R. Tolkien", 1937, publisher_id))
#commit changes
conn.commit()
---------------------------------------------
# Query join
cur.execute("""
SELECT b.title, b.author, p.name AS publisher
FROM books b
JOIN publishers p ON b.publisher_id = p.id
""")
print(cur.fetchall())
conn.close()
-----------------------------------------------
#output
[('The Hobbit', 'J.R.R. Tolkien', 'George Allen & Unwin')]
```

Data management

data models:

- NoSQL: no SQL → not only SQL
  - pros: no schemas → flexibility, scalability, treat of unstructured data
  - cons: storage capacity, performance trade-offs, data chaos
  - document stores: data are stored as a JSON (string)
    - query can be a more difficult task
    - examples: mongoDB, CouchDB – create own index tables for effectivity
  - key-value pairs: for fast query (example: RedisDB, DynamoDB)
  - graph databases: concentrate on relations (nodes and edges) (Examples: Neo4j, Amazon Neptune, ArangoDB)

Data management

data models:

- NoSQL: no S
  - pros: no sc
  - cons: stora
  - document s
    - query ca
    - example
  - key-value p
  - graph datab
    (Examples: N

```
#install mongodb-image and pymongo
docker run -d --name mongodb -p 27017:27017 -v ~/my_data/mongo:/data/db mongo:7.0
pip install pymongo
------------------------------------
#runs a service in the background – start/stop it
docker start/stop mongodb
--------------------
#install mongodb and pymongo
from pymongo import MongoClient

# 1. Connect to local MongoDB
client = MongoClient("mongodb://localhost:27017/")

# 2. Create (or use existing) database
db = client["library"]

# 3. Create collections
publishers = db["publishers"]
books = db["books"]

# 4. Clear old data (for demo purposes)
publishers.delete_many({})
books.delete_many({})
```

Data management

data

- No

```
# 5. Insert publishers
pub_allen = publishers.insert_one({"name": "George Allen & Unwin"}).inserted_id
pub_secker = publishers.insert_one({"name": "Secker & Warburg"}).inserted_id
pub_chatto = publishers.insert_one({"name": "Chatto & Windus"}).inserted_id
pub_little = publishers.insert_one({"name": "Little, Brown and Company"}).inserted_id

# 6. Insert books with publisher references
books.insert_many([
    {"title": "The Hobbit", "author": "J.R.R. Tolkien", "year": 1937, "publisher_id": pub_allen},
    {"title": "The Fellowship of the Ring", "author": "J.R.R. Tolkien", "year": 1954, "publisher_id": pub_allen},
    {"title": "Nineteen Eighty-Four", "author": "George Orwell", "year": 1949, "publisher_id": pub_secker},
    {"title": "Brave New World", "author": "Aldous Huxley", "year": 1932, "publisher_id": pub_chatto},
    {"title": "The Catcher in the Rye", "author": "J.D. Salinger", "year": 1951, "publisher_id": pub_little},
])

# 7. Query: find all books and join publisher info manually
for book in books.find():
    publisher = publishers.find_one({"_id": book["publisher_id"]})
    print(f"{book['title']} by {book['author']} "
          f"was published by {publisher['name']} in {book['year']}.")
```

Data management

data storage:

- data warehouse:
  - stores historical data → structured, cleaned, integrated
  - optimized for analysis, reporting → few but very large, complex queries
  - ETL (extract, transform, load)
  - OLAP (online analytic processing)
  - examples: historical sales data, stock price data
  - solutions: Amazon Redshift, Google BigQuery

Data management

data storage:

- database:
  - stores current, operational data.
  - used for day-to-day transactions (insert, update, delete).
  - optimized for fast read/write, many small simple queries
  - example: an e-commerce database storing users, orders, payments
  - OLTP (Online Transaction Processing)
  - solutions: MySQL, PostgreSQL, Oracle DB, MongoDB, DynamoDB

Data management

data storage:

- data lake:
  - stores data in raw form → future-proof
  - application goal not needed to be specify
  - cheap and scalable storage (HDFS, Amazon S3, Azure Data Lake Storage, Google Cloud Storage)
  - schema-on-read solutions (ELT, extract, load, transform) → processing engines (Kafka, TensorFlow, Athena)
  - example: log storage, IoT data storage

Data management

data sampling: collecting all data usually not feasible

- nonprobability sampling → focus on data collection; prone to selection bias
  - convenience sampling: take all what is available
  - snowball sampling: start with a small set, and follow its descendants
  - judgement sampling: experts say what to collect
  - quota sampling: collect the same number of data from different areas (like age groups)
  - example of bias: psychology experiments (university students), sentiment analysis (wordy people), medical data (unsuccessful treatments)

Data management

data sampling:

- probability sampling → needs a data model → biased?
  - stratified sampling: collect the same amount of data per class
  - weighted sampling: take into account data scaled with probability
    - in choice: class A 25%, class B 75% → consider A samples 3 times
    - in training: give weight for underrepresented classes (see later)
  - reservoir sampling: keep the first k sample, and replace it randomly
  - importance sampling

$$E_P[x]=\sum_x x\ P(x)=\sum_x x\ P\frac{(x)}{Q}(x)Q(x)=E_Q[x\,P\frac{(x)}{Q}(x)]$$

Data management

data augmentation:

- if there are not enough data, or not representative enough, we can transform data to generate new ones (symmetries)
  - rotation, scaling
  - perturbation, noising
  - texture or style change
- data generation: simulations in artificial environment (e.g. self-driving cars)

Data management

labelling:

- for classification we should collect examples in different classes
- problems:
  - how unambiguous are the labels? → multi-label classes
  - how precise are the labels? → no database is 100% correct!
  - what to do with missing labels? → label functions, semi-supervised, etc.
- hand labelled data, naturally labelled data (context is given), automatically generated data

Monitoring and evaluation

Monitoring and evaluation

How can we assess the success of the ML results?

- not unique! depends on the problem on hand
  - example: percentage of correctly classified images
  - if the probability of belonging to class A is 99%, then the simple method saying that *all* elements belong to A is 99% accurate!
  - clearly not good, if the task is to find the 1% not-A class!

Monitoring and evaluation

- baselines: how accurate are simple methods and other approaches?
  - random baseline: choose class randomly → 1/N probability for balanced
  - zero rule: all belonging to one class → high probability for inbalanced
  - simple heuristic → use a simple proxy (feature)
  - human experts → how well do humans perform?
  - other ML methods

Monitoring and evaluation

- confusion matrix
  - correct classes vs. estimated classes
  - works for multi-class evaluation
  - in probabilistic sense joint probability: $C_{ij} = P(\text{predicted}_i, \text{actual}_j)$

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.510	FP=0.038	PP=0.548
Predicted No	FN=0.032	TN=0.420	PN=0.452
Actual Total	AP=0.542	AN=0.478	1.0

Monitoring and evaluation

- correctness measures for 2-class evaluation
  - *accuracy*: prob. of choosing the correct class: (TP+TN)
  - *precision*: conditional for Predicted Yes: $P(\text{actual}_P|\text{predicted}_P)=\frac{TP}{PP}$
  - *recall*: conditional for Actual Yes: $P(\text{predicted}_P|\text{actual}_P)=\frac{TP}{AP}$
  - *type I error*: probability of false alarm: $P(\text{predicted}_P|\text{actual}_N)=\frac{FP}{AN}$
  - *type II error*: probability of missing signal: $P(\text{predicted}_N|\text{actual}_P)=\frac{FN}{AP}$

Monitoring and evaluation

- controlled case → measure does not matter
  - probability of class A or B is 50%, predict symmetrically
  - accuracy=98%
  - for class A: precision=98%, recall=98%, type II error=2%
  - for class B: precision=98%, recall=98%, type I error=2%

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.49	FP=0.01	PP=0.5
Predicted No	FN=0.01	TN=0.49	PN=0.5
Actual Total	AP=0.5	AN=0.5	1.0

Monitoring and evaluation

- extreme cases
  - classify everything as class A, but probability of class A is 99%
  - accuracy=99%
  - for class A: precision=99%, recall=100%, type II error=0%
  - for class B: precision=?, recall=0%, type I error=100%

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.99	FP=0.01	PP=1.0
Predicted No	FN=0.0	TN=0.0	PN=0.0
Actual Total	AP=0.99	AN=0.01	1.0

Monitoring and evaluation

- extreme cases
  - classify everything randomly, but probability of class A is 98%
  - accuracy=50%
  - for class A: precision=98%, recall=50%, type II error=50%
  - for class B: precision=2%, recall=50%, type I error=50%

	Actual Yes	Actual No	Predicted Total
Predicted Yes	TP=0.49	FP=0.01	PP=0.5
Predicted No	FN=0.49	TN=0.01	PN=0.5
Actual Total	AP=0.98	AN=0.02	1.0

Monitoring and evaluation

- effect of a threshold on the prediction
  - in lot of cases Yes/No decision comes from comparing a signal with a threshold → e.g. smoke detector signal vs. danger level
  - ROC (receiver operating characteristics)
    - $P(\text{predicted}_P | \text{actual}_P)$ vs. $P(\text{predicted}_P | \text{actual}_N)$
    - recall vs. type I error
    - correct alarm probability vs. false alarm probability
  - AUC: area under curve

Monitoring and evaluation

- effect of a threshold on the prediction

Data modelling

Data modelling

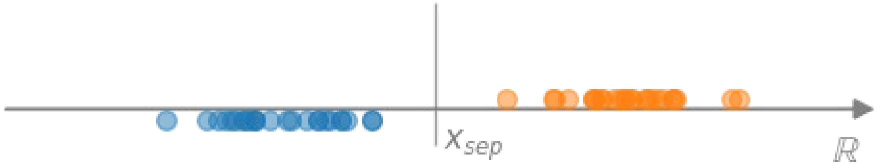
- task: find out the selective/descriptive features
- depends both on the dataset and the context
- selective: constant on a class, descriptive: changes within a class
  class depends on the context
- finding the selective coordinates is the main challenge
- we start with simple 1D classification problems, and generalize to several dimensions

One dimensional dataset classification

One dimensional dataset

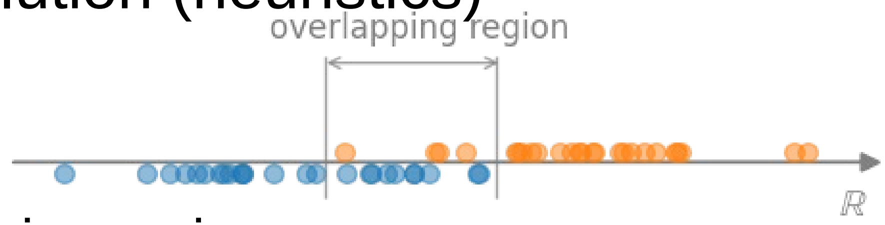
- Simplest basic case: characterization of one-dimensional data
  - data → $x \in \mathbb{R}$ (numbers for analytic characterization)
- Separation of two 1D sets
  - assume that in given sections belong (approximately) to a single class
  - basic case: one section → A, other section → B
  - if the two sets are separated (hard margin)
  - overlapping case? → no perfect solution

$$\begin{cases} x \in A \Leftrightarrow x < x_{sep} \\ x \in B \Leftrightarrow x > x_{sep} \end{cases}$$



One dimensional dataset

- overlapping regions → no perfect solution (heuristics)
  - choose x_{sep} somewhere in the overlapping region (eg. midpoint)



  - define en error function in the overlapping region, e.g.
  - p=1 (“hinge loss”) → median, p=2 → mean $L_p(x_{sep}) = \sum_{x \in \text{overlap}} |x - x_{sep}|^p$
  - we can weight the points

$$0 = \frac{d}{dx_{sep}} \sum_{i=1}^{N} (x_i - x_{sep})^2 = 2 \sum_{i=1}^{N} (x_{sep} - x_i) \Rightarrow x_{sep} = \frac{1}{N} \sum_{i=1}^{N} x_i$$

$$\sum_{i=1}^{N} |x_i - x_{sep}| = \sum_{x_i > x_{sep}} (x_i - x_{sep}) + \sum_{x_i < x_{sep}} (x_{sep} - x_i) = (N_< - N_>) x_{sep} + \sum_{x_i > x_{sep}} x_i - \sum_{x_i < x_{sep}} x_i \Rightarrow N_< = N_>$$

One dimensional dataset

- Another approach: treat the problem as a regression
  - define a function $f:\mathbb{R}\to\mathbb{R}$, that $\begin{Bmatrix} f(x)<0 & \Leftrightarrow & x\in A \\ f(x)>0 & \Leftrightarrow & x\in B \end{Bmatrix}$
  - a classification can always be translated to regression
  - simplest: linear relation f(x)=ax+b
  - probabilistic interpretation: $p(x)=\frac{e^{f(x)}}{1+e^{f(x)}}\in[0,1],\quad p(0)=0.5$

    logistic map, probability of belonging to class B
  - loss function: $L(a,b)=\sum_x \left(p(x)-I_{x\in B}\right)^2,\qquad I_{x\in B}=1 \text{ if } x\in B, \text{ else } 0$

One dimensional dataset

- Another approach: trea
  - define a function $f:$ I
  - a classification can alw
  - simplest: linear relatio
  - probabilistic interpreta

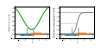
    logistic map, probability of belonging to class B
  - loss function: $L(a,b) = \sum_x \left(p(x) - I_{x\in B}\right)^2, \quad I_{x\in B} = 1 \text{ if } x \in B, \text{ else } 0$

```
# Python function minimization
import numpy, matplotlib
def P(p, x):
	a,x0=p
	return np.exp(a*(x-x0))/(1+np.exp(a*(x-x0)))

def loss(p, A, B):
	loss = np.sum([P(p,a)**2 for a in A])
	loss += np.sum([(P(p,b)-1)**2 for b in B])
	return loss

p0 = [1, 1]  # initial guess
params = (A,B)  # values of a, b
res = minimize(loss, p0, args=params)

a,x0 = res.x
```

One dimensional dataset

- Another approach: treat the problem as a regression
  - logistic map, probability of belonging to class B

$$p(x)=\frac{e^{ax+b}}{1+e^{ax+b}}$$



Probability and information theory basics

Information theory basics

Practical **probability theory** (adapted to ML):

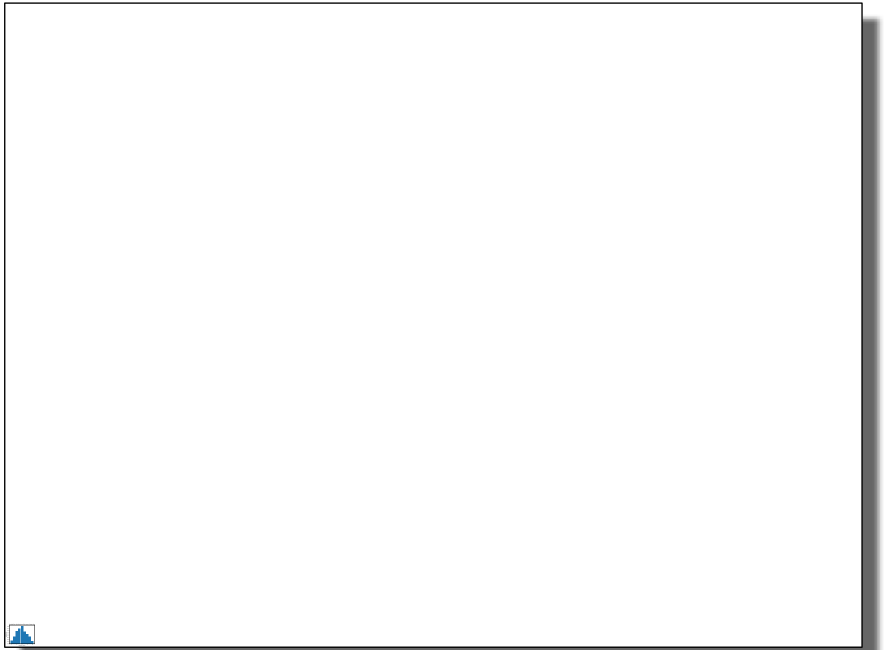
- possible outputs of an event: sample space $\omega \in \Omega$
- **features** ($\rightarrow$ random variables): $X : \Omega \rightarrow \mathbb{R}^n$ functions, $x = X(\omega)$
- observe a list of events: $(\omega_1, \omega_2, \ldots, \omega_N)$
- **probability**: $P(X \in U \subset \mathbb{R}^n) = \frac{1}{N} \sum_{i=1}^{N} \delta_{X(\omega_i) \in U}$ $\rightarrow$ **histogram**
- **distribution**: $p_X(x) = P(X \in \{x\})$
- *probability density:* $p_X(x) = \frac{P(X \in [x, x+dx])}{|dx|}$

Indicator function

$$\delta_b = \begin{cases} 1 \text{ if } b = True \\ 0 \text{ if } b = False \end{cases}$$

example of 2 dice thrown N times

```
# generate data
Ndata = 1000
data = np.random.randint(1,6,size=(Ndata,2))
feature = data[:,0] + data[:,1]

# measure histogram
fmax = int(max(feature))
fmin = int(min(feature))
bins = np.zeros(fmax-fmin+1)
for f in feature:
    bins[int(f-fmin)] += 1
bins/=Ndata

# show result
fig,ax = plt.subplots()
ax.bar(np.arange(fmin,fmax+1),bins)
ax.set_xlabel('feature value', fontsize=14)
ax.set_ylabel('probability', fontsize=14)
```



Information theory basics

Joint probability distributions

- the features can have several components → $XY=(X,Y):\Omega\rightarrow\mathbb{R}\times\mathbb{R}$
- their distribution is called joint probability distribution: $p_{XY}(x,y)$
- if we are interested only on one component: $p_X(x)=\sum_{y\in Y} p_{XY}(x,y)$
- for independent features the value of x does not depend on the value of y $p_{XY}(x,y)=p_X(x)\,p_Y(y)$

$$\boxed{p_X(x)=\frac{1}{N}\sum_{i=1}^{N}\delta_{X(\omega)=x}=\frac{1}{N}\sum_{i=1}^{N}\sum_{y\in Y(\Omega)}\delta_{X(\omega)=x;Y(\omega_i)=y}=\sum_{y\in Y(\Omega)}p_{XY}(x,y)}$$

Information theory basics

Information theory: statistical relation of measurements

- assume we have measured a lot $N \to \infty$
- allows simplifications → Stirling formula: $\ln(n!) \approx n \ln n + \ldots$
- measure of the uncertainty / unpredictability / impurity of a probability distribution → **Shannon entropy**

Information theory basics

Shannon entropy: measure of variability of a distribution

- perform N measurements with n possible outputs (bins)
- count the number of results in each bin: $\{N_1, N_2, \ldots, N_n\}$
- how many realizations is possible for a given $\{N_1, N_2, \ldots, N_n\}$?
- Shannon entropy: log of this number over N
- *equivalent to the entropy concept in physics: number of microstate representation of a macrostate*

Information theory basics

Shannon entropy:

- take n bins, N independent measurements
- we obtain $\{N_1, N_2, \ldots, N_n\}$ → probability $p_i = \dfrac{N_i}{N}$
- how many ways can this distribution be realized? *(measurements within a bin are indistinguishable)*
- binomial – multinomial expression

$$\{N_1, N_2\} \rightarrow \binom{N}{N_1} = \frac{N!}{N_1!\,N_2!} \qquad \{N_1, N_2, \ldots, N_n\} \rightarrow \frac{N!}{N_1!\,N_2!\ldots N_n!}$$

Information theory basics

Shannon entropy:

- information theory: $N_i \rightarrow \infty$ for all parts
- Stirling formula, using $N = \sum_{i=1}^{n} N_i$

$$\log \frac{N!}{N_1! N_2! \ldots N_n!} \approx N \log N - \sum_{i=1}^{n} N_i \log N_i = -N \sum_{i=1}^{n} p_i \log p_i$$

- Shannon entropy:

$$H = \lim_{N_i \to \infty} \frac{1}{N} \log_2 \frac{N!}{N_1! N_2! \ldots N_n!} = -\sum_{i=1}^{n} p_i \log_2 p_i$$

Information theory basics

Shannon entropy: $H = -\sum_{i=1}^{n} p_i \log_2 p_i$

- always positive or zero $H \geq 0$
- minimal for a pure sample $p_1 = 1, p_{i>1} = 0 \Rightarrow H = 0$
- maximal for uniform sample $p_i = \frac{1}{N} \Rightarrow H = \log_2 N$
- convex

$$\text{if} \quad E = \sum_i E_i P_i \quad \text{fixed} \quad \Rightarrow \quad p_i \sim e^{-\beta E_i}$$

Information theory basics

Other entropy formulae:

- we assumed independence of measurement bins
  what if they are not (non-additive systems)
- general definition using properties
  (positive, convex)
- examples:
  - Rényi-Tsallis entropy $H_\alpha = \frac{1}{1-\alpha} \log\left(\sum_{i=1}^{n} p_i^\alpha\right)$
  - Gini impurity $G = 1 - \sum_{i=1}^{n} p_i^2$

Information theory basics

Shannon entropy and amount of information

- we have n events with probabilities $\{p_1, p_2, \dots, p_n\}$
- define **information** as the **minimal number of questions** needed to find out which one happened? → *twenty questions game*
- **answer**: Shannon entropy!
  - one question → one "bit" of information
  - Shannon entropy = information content
  - for uniform probabilities logarithmic search, in general Huffmann coding

Information theory basics

Shannon entropy and amount of information – technical slide

- coding: a binary code corresponds an interval in [0,1)

$$(b_1, b_2, \dots, b_k) \Rightarrow [\sum_{i=1}^{k} b_i 2^{-b_i}, \sum_{i=1}^{k} b_i 2^{-b_i} + 2^{-k}]$$

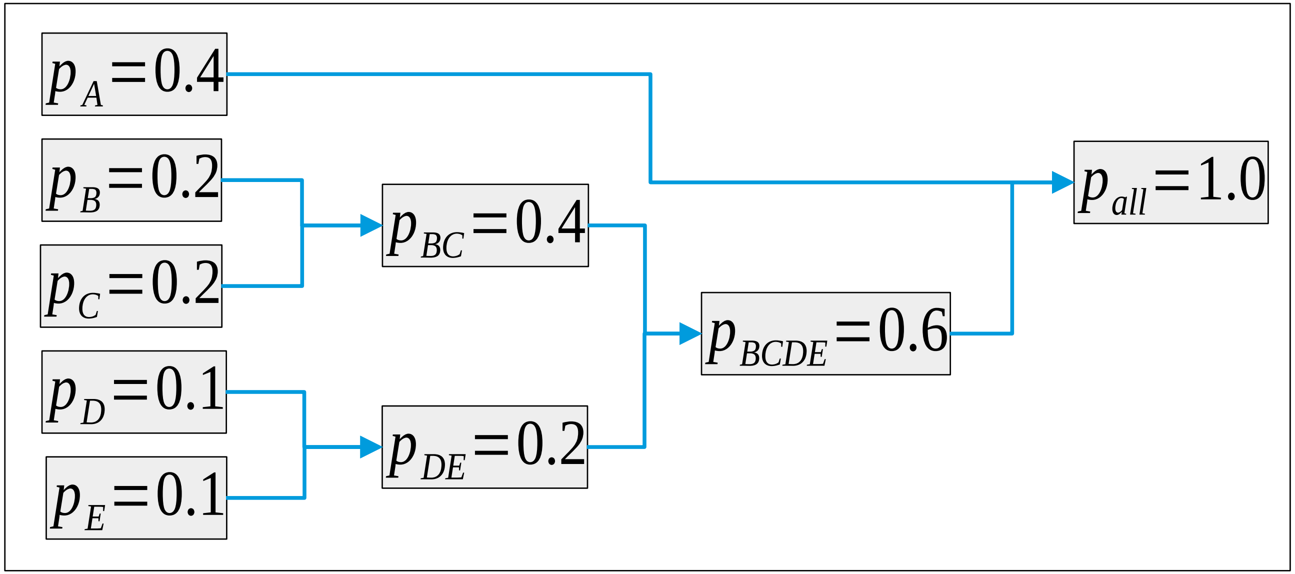
$$\boxed{101 \Rightarrow (1,0,1) \Rightarrow [\frac{1}{2}+\frac{1}{8}, \frac{1}{2}+\frac{1}{8}+\frac{1}{8}] = [\frac{5}{8}, \frac{6}{8}]}$$

- can be placed non-overlapping if $\sum_{i=1}^{k} 2^{-l_i} \leq 1$ (Kraft–McMillan theorem)
- choose code lengths $l_i = \lfloor -\log_2 p_i \rfloor$ $\rightarrow$ non-overlapping
- average code length $H \leq \sum_{i=1}^{n} p_i l_i < H+1$

Information theory basics

Shannon entropy and amount of information

- example for Huffmann coding
- average questions: 0.4+3x0.6=2.2
- Shannon entropy: 2.12



Information theory basics

Joint probability, entropy and mutual information

- if states depend on two variables, the joint probability $p(x,y)$
- for independent variables $p(x,y)=p(x)p(y)$
- the corresponding Shannon entropy

$$H(X,Y)=-\sum_{x\in X,y\in Y}^{n} p(x,y)\log_2 p(x,y) \rightarrow H(X)+H(Y)$$

- measure of independence: mutual information

$$I(X,Y)=H(X)+H(Y)-H(X,Y)$$

Joint probability, entropy and mutual information

- example: 2 bits

```
# Shannon entropy function
def SH(p):
    return -p*np.log2(p+1e-10)

pX = np.sum(p0, axis=1)
pY = np.sum(p0, axis=0)
pXY = np.reshape(p0, -1)
allH = []
for p in [pXY,pX,pY]:
    H = 0
    for pi in np.reshape(p, -1):
        H += SH(pi)
    allH.append(H)
IXY = allH[1] + allH[2] - allH[0]
```

Joint probability, entropy and mutual information

- example: 2 bits

```
# Shannon entropy functio
def SH(p):
    return -p*np.log2(p+1e-

pX = np.sum(p0, axis=1)
pY = np.sum(p0, axis=0)
pXY = np.reshape(p0, -1)
allH = []
for p in [pXY,pX,pY]:
    H = 0
    for pi in np.reshape(p, -
        H += SH(pi)
    allH.append(H)
IXY = allH[1] + allH[2] - allH[0]
```

```
# example: independent case

p0=[[ ¼, ¼ ], [¼, ¼]]

pX = [½, ½]
pY = [½, ½]

# note that pXY = pX*pY

HX = 1
HY = 1
HXY = 2
IXY = 0
```

Information theory basics

Joint probability, entropy and mutual information

- example: 2 bits

```
# Shannon entropy functio
def SH(p):
    return -p*np.log2(p+1e-1

pX = np.sum(p0, axis=1)
pY = np.sum(p0, axis=0)
pXY = np.reshape(p0, -1)
allH = []
for p in [pXY,pX,pY]:
    H = 0
    for pi in np.reshape(p, -1
        H += SH(pi)
    allH.append(H)
IXY = allH[1] + allH[2] - allH[0]
```

```
# example: independent case

p0=[[ ¼, ¼ ], [¼, ¼]]

pX = [½, ½]
pY = [½, ½]

# note that pXY = pX*pY

HX = 1
HY = 1
HXY = 2
IXY = 0
```

```
# example: not independent case

p0=[[ 1/3, 1/6 ], [¼, ¼]]

pX = [½, ½]
pY = [7/12, 5/12]

# note that pXY not = pX*pY

HX = 1
HY = 0.98
HXY = 1.96
IXY = 0.02
```

Information theory basics

Probability of the observed sample

- we randomly choose case ***i*** with probability q_i
- what is the probability that we observe N_i events from class ***i***?
- logic of the result:
  - first we choose event 1, next event 2, etc. $\rightarrow \prod_{i=1}^{n} q_i^{N_i}$
  - number of such events is $\dfrac{N!}{N_1!N_2!\dots N_n!}$
  - probability $P=N!\prod_{i=1}^{n}\dfrac{q_i^{N_i}}{N_i!}$

Information theory basics

Probability of the observed sample

- logarithm of the result in the N → ∞ case

$$\log P = N \log N + \sum_{i=1}^{n} \left(N_i \log q_i - N_i \log N_i \right) = N \sum_{i=1}^{n} p_i \log \frac{q_i}{p_i}$$

- Kullback-Leibler divergence: $D(p\|q) = \frac{1}{N} \ln P$

$$\boxed{D(p\|q) = \sum_{i=1}^{n} p_i \ln \frac{q_i}{p_i}}$$

- interpretation: “distance” of probability distributions

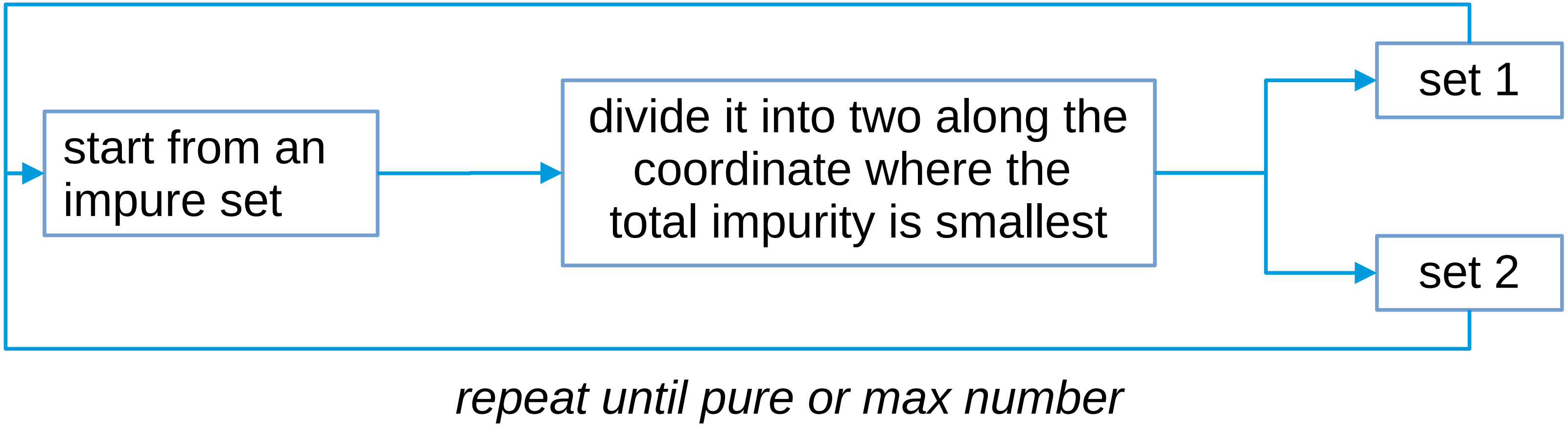
Decision trees,
ensemble improvements,
K-nearest neighbour (KNN) method

Tree based methods

- The **generalization to multiple dimensions** is not simple
  - in 1D the coordinates are unique
  - in multidimensional case we can optimize the coordination → features
  - simplest case: use original coordinates
  - usually the original coordinates are not equally useful
    (c.f. image → x coord separates more)
  - choose that coordinate that leads to the smallest total impurity!

Tree based methods

- Decision tree (DT)



- always converges, but may result in fragmented divisions

Tree based methods

- Decision tree (DT) example
  - two 2D sets
- pros:
  - simple
  - interpretable
- cons:
  - not robust enough
    (→ *irregular shape)*

Tree based methods

- Decision tree (DT) example
  - two 2D sets
- pros:
  - simple
  - interpretable
- cons:
  - not robust enough

    (→ *irregular shape)*

    use fixed depth!
  - accuracy*: train: 100%, test: 92.5%*

Tree based methods

- Decision tree (DT) technical solution in Python → github

```
# Decision Tree example
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

# 1. get or generate data and labels


# 2. Fit a decision tree
clf = DecisionTreeClassifier()
clf.fit(data, labels)

# 3. use for prediction
pred_labels = clf.predict(other_data)

#4. confusion matrix
cm = confusion_matrix(labels, pred_labels)
```

Ensemble methods

Ensemble methods

- Decision tree → too specific for a sample
- Possible solution: use ensemble of methods
- ways of improvement
  - bagging (Bootstrap AGGregatING) → bagging, random forest
  - boosting → adaptive boosting, gradient boosting
  - other methods → extra trees, isolated forests, oblique trees

Bagging

- Idea:
  - train several models in slightly different datasets
  - each dataset contains the same number of data from the database, but some of them are missing, some appear multiple times
  - combine the results → regression: average, classification: majority vote
- Benefits:
  - lowers variance
  - increases robustness

Bagging

- Code: github

```
# Bagging example
from sklearn.ensemble import BaggingClassifier
from sklearn.tree import DecisionTreeClassifier
from sklearn.metrics import confusion_matrix

# bagging based on decision tree
base_tree = DecisionTreeClassifier()
clf = BaggingClassifier(
    estimator=base_tree,
    n_estimators=50,          # number of trees
    max_samples=0.8,          # each tree sees 80% of data
    bootstrap=True,           # sampling with replacement
    random_state=42
)
clf.fit(X,y)

# 3. use for prediction
pred_labels = clf.predict(other_data)

#4. confusion matrix
cm = confusion_matrix(labels, pred_labels)
```

Random forest

- **Idea**: in addition vary the used parameters, too
  - train several models in slightly different datasets
  - each model uses selected data and selected parameters
  - combine the results → regression: average, classification: majority vote
- **Benefits**:
  - more robust than simple bagging
  - can be used to rank parameters by their significance, i.e. which contributed more to the final decision

Random forest

- 2D case
- accuracy: 94%

```
# Random Forest example
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import confusion_matrix
# bagging based on decision tree

# Train Random Forest
clf = RandomForestClassifier(
        n_estimators=100, # number of trees
        max_depth=None, # let trees grow fully
        random_state=42  # random generator seed
)
clf.fit(X, y)

# 3. use for prediction
pred_labels = clf.predict(other_data)

#4. confusion matrix
cm = confusion_matrix(labels, pred_labels)
```

Boosting

- Idea:
  - calculate a series of decision trees
  - each focuses on the points where the previous one fails
  - combine weighted sum of all of these models
- Adaptations: AdaBoost, Gradient Boosting (GBM), XGBoost, …

Boosting

- 2D case
- accuracy: 94%

```
# AdaBoost example
from sklearn.metrics import confusion_matrix
from sklearn.ensemble import AdaBoostClassifier

# boosting based on decision tree

# Train AdaBoost model
boost = AdaBoostClassifier(
        estimator=DecisionTreeClassifier(max_depth=2),
        n_estimators=50,
        learning_rate=1.0,
        random_state=42)
boost.fit(X, y)

# 3. use for prediction

pred_labels = boost.predict(other_data)

#4. confusion matrix
cm = confusion_matrix(labels, pred_labels)
```

k-nearest neighbour (KNN) model

KNN

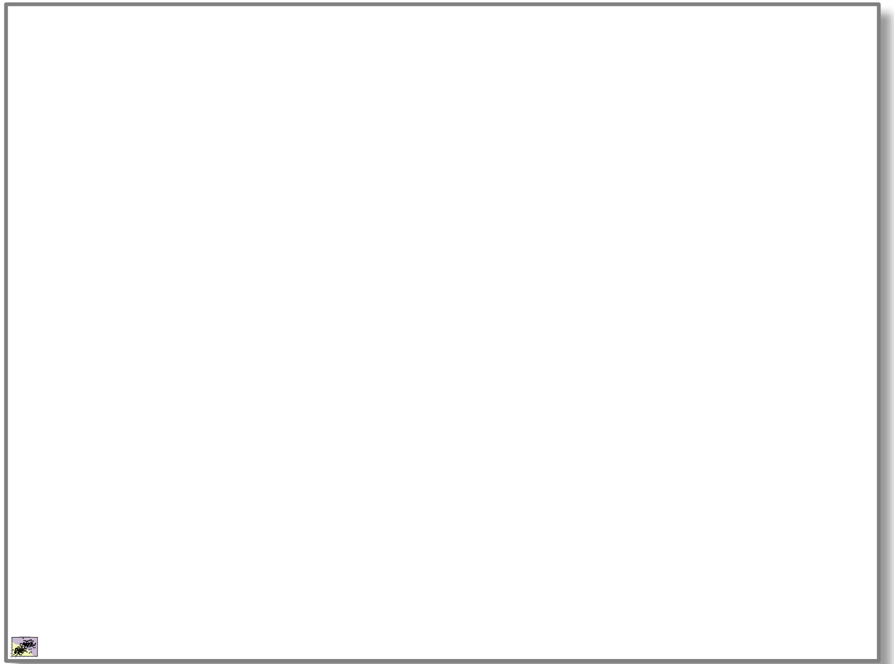
- a different idea: similar points are close in the feature space
- for a good guess on the class peek the neighbouring points
- algorithm:
    - keep all points in the training set
    - new point: compute all the distances
      → needs a distance measure
    - keep first k closest points
    - estimated class: majority vote

KNN

- ✅ pros
  - simple, intuitive, no training phase
  - naturally handles multi-class problems
- ❌ cons:
  - slow at prediction time (needs to compare with all samples)
  - sensitive to irrelevant features and scaling (c.f. distance measure)
  - struggles in high dimensions (“curse of dimensionality”)

KNN

- in code



```
# Random Forest example
from sklearn import svm
from sklearn.neighbors import KNeighborsClassifier

# Train a 3-NN classifier
knn = KneighborsClassifier(n_neighbors=5)
knn.fit(X,y)

# Predict on the test set
y_pred = knn.predict(X_test)

# Evaluate
acc = accuracy_score(y_test, y_pred)

→ acc = 97.5%
```

Estimate relevance of features

Relevance of features

- not all features contribute equally to the result – to reduce the number of features we shall asses their relevance
- idea: measure the change in the result when change the feature
- relevance measures (the most used ones)
  - feature importance (only for random forest)
  - permutation importance, feature masking
  - mutual information
  - in deep neural networks Gradient-weighted Class Activation Mapping (Grad-CAM)

Relevance of features

- we will demonstrate the feature relevance measures in a 2-class 20 dimensional dataset with 2000 elements

```
# make mulitdimensional dataset
from sklearn.datasets import make_classification
from sklearn.model_selection import train_test_split

ndim = 20
Nsamples = 2000

X, y = make_classification(n_samples=Nsamples, n_features=20,
n_classes=2, random_state=42)

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
random_state=42)
```

Feature importance

- works only for random forest
- idea:
  - measure the impurity (e.g. Gini impurity) of all nodes before and after a decision
  - if we use a feature for a decision, measure the impurity decrease

$$\text{Importance}(f) = \sum_{\text{nodes using } f} \text{Impurity decrease at that node}$$

  - average over the forest

$$\text{Importance}_{forest}(f) = \frac{1}{N_{trees}} \sum_{t=1}^{N_{trees}} \text{Importance}_t(f)$$

Feature importance

- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier

```
# make mulitdimensional dataset
from sklearn.ensemble import RandomForestClassifier

# Creating a Random Forest classifier
clf = RandomForestClassifier(n_estimators=100, random_state=42)

# Training the classifier on the training data
clf.fit(X_train, y_train)
```

Feature importance

- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier
- evaluate the result

```
# evaluate result
from sklearn.metrics import confusion_matrix

# Making predictions on the testing data
y_pred = clf.predict(X_test)

# confusion matrix
cm = confusion_matrix(y_test, y_pred)

accuracy = np.trace(cm) / np.sum(cm)

→ accuracy = 93.5%
```

Feature importance

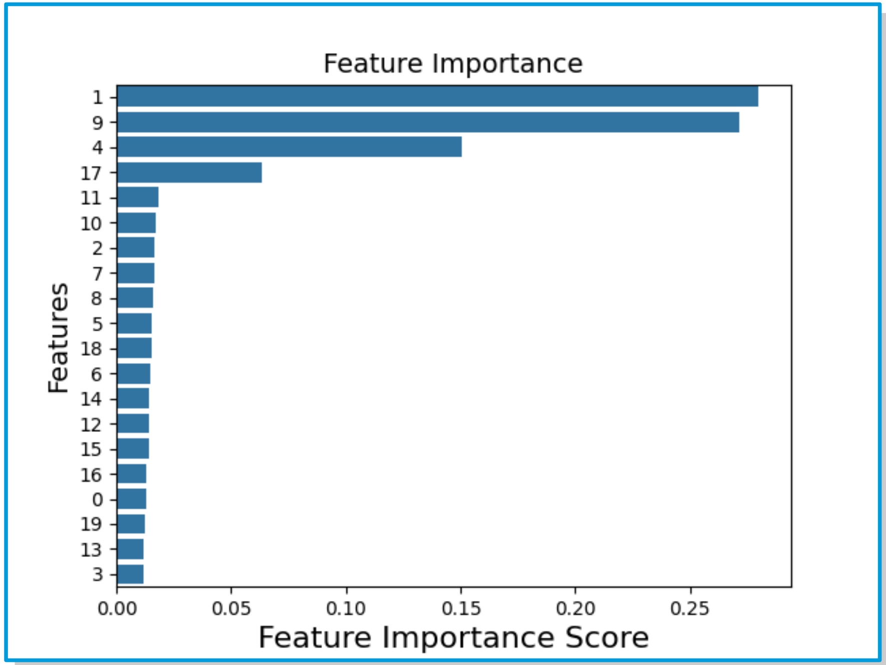
- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier
- evaluate the result
- get feature importance

```
feature_importance =clf.feature_importances_

→np.array([0.01291599, 0.27999269, 0.01658218,
0.01157727, 0.15054909, 0.01555601, 0.0150478 ,
0.01637177, 0.01582075, 0.27170634, 0.01723006,
0.01812296, 0.01401398, 0.01165928, 0.01404261,
0.01398676, 0.01323008, 0.0636087 , 0.01551626,
0.01246943])
```

Feature importance

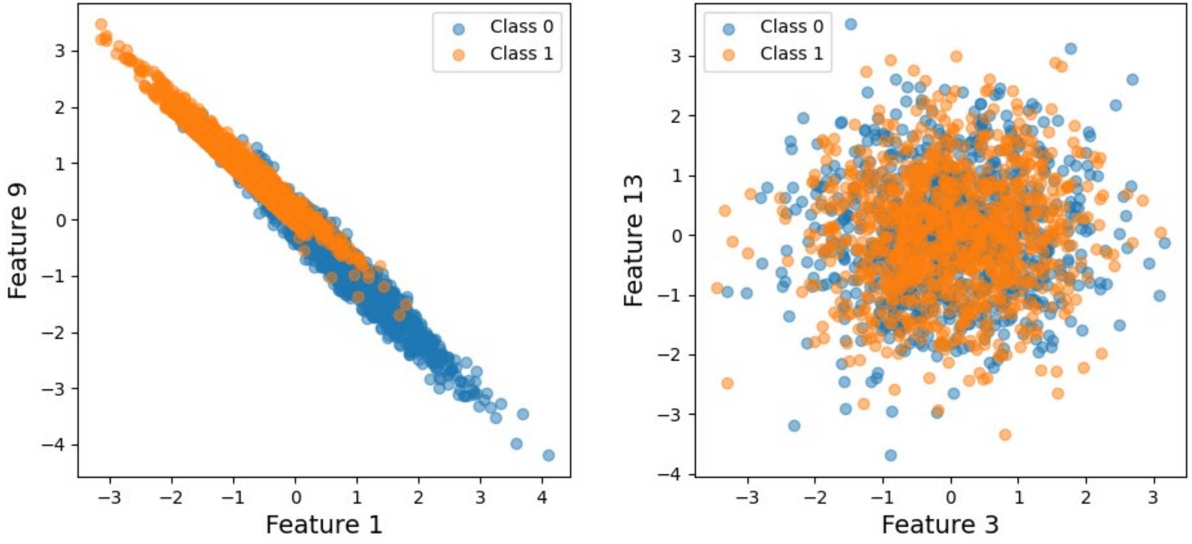
- example: create a 20 dimensional dataset with 2000 elements
- train a random forest classifier
- evaluate the result
- get feature importance
- visualize it

```
sorted_idx = np.argsort(feature_importance)[::-1]

plt.figure()
sns.barplot(
        x=feature_importance[sorted_idx],
        y=np.arange(len(sorted_idx)),
        orient='h')
plt.yticks(np.arange(len(sorted_idx)),sorted_idx)
```



Feature importance

- example: create a 20 dimensional dataset with 2000 elements
- most (1,9) and least (13,3) important features



Permutation importance

- works for all methods
- idea:
  - train the model, measure the performance – baseline
  - for each feature permute the values among samples
  - measure the performance again
  - relevance of the feature: drop in performance

$$\mathrm{Importance}(f) = \mathrm{Score}_{original} - Score_{permuted}(f)$$

  - instead permutation we can use a constant value (e.g. average), or a random value $\rightarrow$ mask that feature

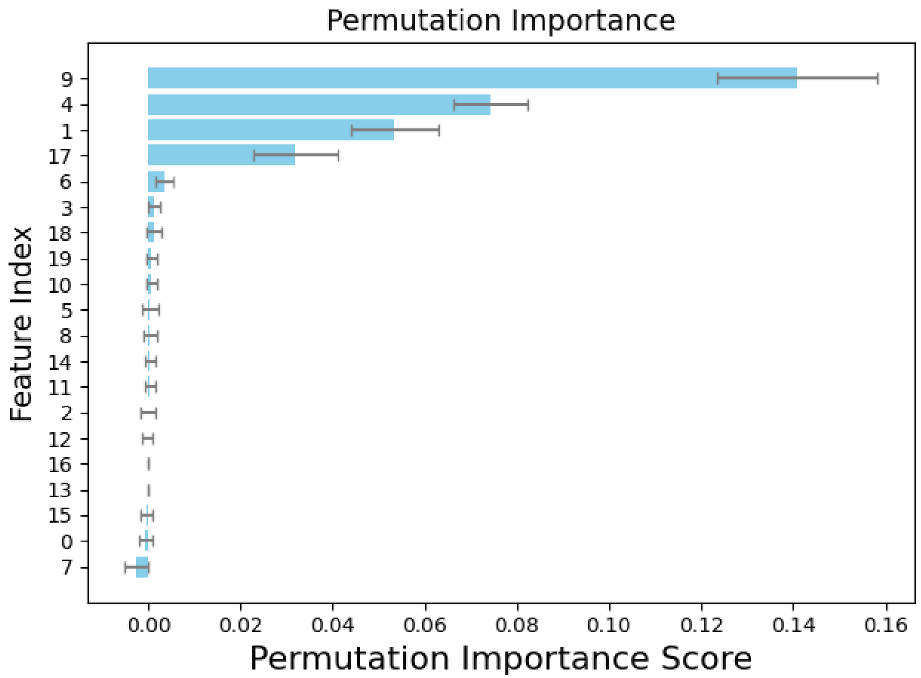
Permutation importance

- get permutation importance in Pyhton

```
# import package
from sklearn.inspection import permutation_importance

# use trained model clf to calculate importance
result = permutation_importance(clf, X_test, y_test,
n_repeats=10, random_state=42)

# read mean and variance of the estimates
importances = result.importances_mean
std = result.importances_std
```

Permutation importance

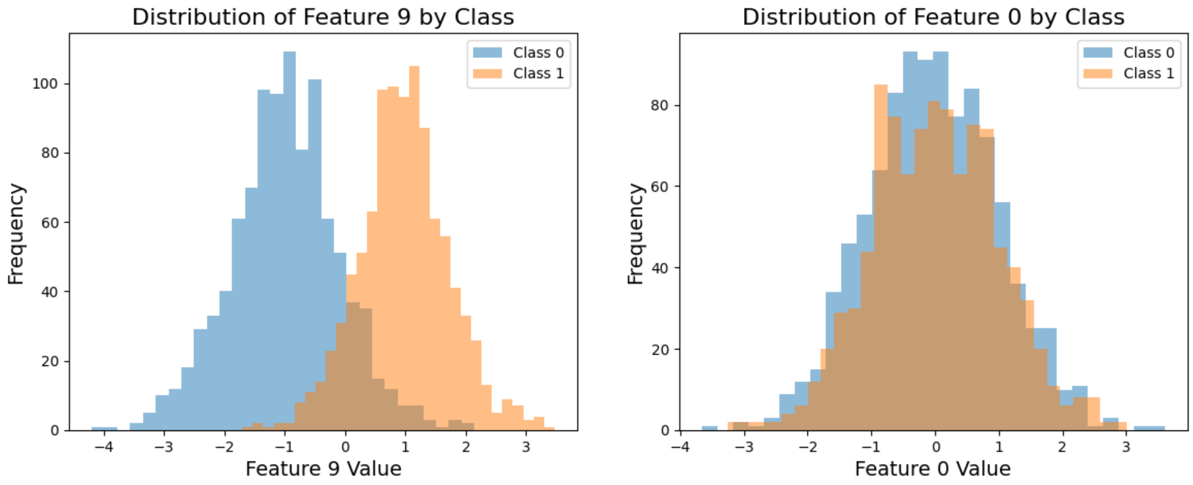
- get permutation importance in Pyhton
- visualize the result
- same features are selected as before, but order is different



Permutation importance

- choose feature 9, and feature 0 to demonstrate the effect
- plot the distribution of the given feature, conditioned for the class (histogram)

$$P(\text{feature value} = f \mid \text{class})$$



Mutual information

- can be used for any models – for any two set, actually
- idea:
  - calculate the joint probability of two sets $p(x \in X, y \in Y)$
  - if x and y are independent, then the joint probability is the product

$$p(x,y) \xrightarrow{\text{independent}} p_X(x)\,p_Y(y)$$

  - measure the dependence (mutual information) by

$$I(X,Y) = \sum_{x \in X, y \in Y} p(x,y) \ln\left(\frac{p(x,y)}{p_X(x)\,p_Y(y)}\right)$$

  - can be used to measure mutual information between features and labels

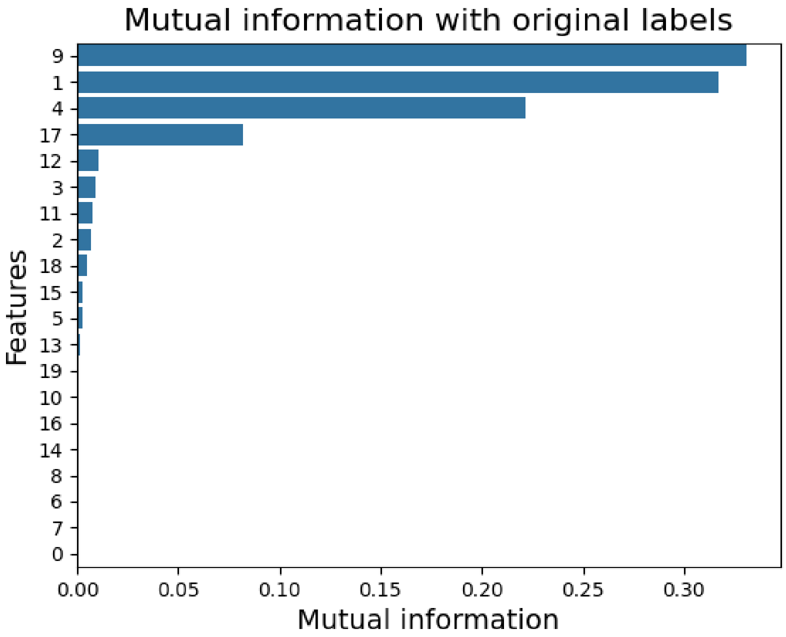
Mutual information

- in Python there is a package to calculate mutual information
- the code

```
# import package
from sklearn.feature_selection import mutual_info_classif, mutual_info_regression

# we can use the original labels, or the model-predicted ones
# characteristic for both the system and the trained model
# y = clf.predict(X)

mi_model = mutual_info_classif(X, y, random_state=42)

mi_model /= np.sum(mi_model)
```

Mutual information

- in Python there is a package to calculate mutual information
- the code
- visualization
- similar result as before



Regression

Regression

Regression is optimization in a continuous parameter space

- Data model → feature optimization:
  - we need a parametric combination of the original variables
  - optimize with respect to some condition (c.f. SVM)
- Continuous value estimation:
  - the task can be to predict a continuous variable
  - classification can be based on probability distribution

Regression

task of regression

- we have data pairs (X, Y) → database
- we are looking for a model that $f(x;\omega)=y, \text{ for all } x,y \in X \times Y$
  - we can not consider all possible functions (too large space)
  - fix a functional space (e.g. polynomials, Gaussians, etc.): $f \in F$
  - the variables omega select the actual function

task of regression

- there is no perfect model! → *philosophical problem …*
  - not enough observation (data)
  - data do not contain all information
  - functional space does not cover all possible effects
- consequence: $f(x;\omega)=y, \text{ for all } x, y \in X \times Y$ can not be satisfied
- what can be achieved: best representation, where the error is the smallest → what is “best”, and what is “error”?

Regression

task of regression – theoretical approach

- not enough information: x→y relation is not unique
- there is a probability to measure a *(data, label)* pair: $\rho(x, y) \in [0,1]$
- for the complete dataset, assuming independent measurements

$$\rho(X, Y) = \prod_{n=1}^{N} \rho(x_n, y_n)$$

- for a given input there can be a lot of outputs: conditional probability (instead of a function) $\rho(Y \mid X) = \frac{\rho(X, Y)}{\rho(X)}$

Regression

Bayes theorem

$$p(A|B)=\frac{p(A,B)}{p(B)}$$

task of regression – theoretical approach

- data model: $\rho(Y|X) \Rightarrow \rho(Y|X;\omega)$
- what is the best parameter choice? → *ensemble of worlds*
- define the “distribution of the parameters” using Bayes theorem

$$p(\omega|X,Y)=\frac{p(\omega,Y|X)}{p(Y|X)}=\frac{p(Y|\omega,X)\,p(\omega|X)}{p(Y|X)}$$

- as the function of the parameter → *likelihood function*
(assuming no restriction to possible parameter values)

$$L(\omega)=p(\omega|X,Y)\sim p(Y|\omega,X)$$

Regression

task of regression – theoretical approach

- best parameter value when $L(\omega)=p(\omega\,|\,X,Y)$ is maximal *(maximum likelihood):*
$$\frac{\partial L}{\partial \omega_i}(\omega_0)=0$$
- *confidence levels*: $\Omega_p=\{\omega \mid L(\omega)\geq L_p=\text{constant}\}$ $\rightarrow$ hypersurfaces
- a confidence level can be characterized with the probability to be inside the volume $p=P(\omega\in\Omega_p)$
    - *for example: the probability that* $\omega\in\Omega_{0.95}$ *is 95% (according to* $L(\omega)$*)*

Regression

task of regression – theoretical approach

$$N(\mu,C)(y)=\frac{1}{\sqrt{\det(2\pi C)}}e^{-\frac{1}{2}(y-\mu)C^{-1}(y-\mu)}$$

- in practice we use **Gaussian** *(central limit theorem)*
  - assume that data points in the database are independent
  - data model gives the expected value
  - with unit correlation matrix → least square method (LSM)

$$\rho(Y\,|\,X;\omega)=\prod_{n=1}^{N}N(f(x_n;\omega),C_n)(y_n)\sim e^{-\frac{1}{2}\sum_{n=1}^{N}(y_n-f(x_n;\omega))C_n^{-1}(y_n-f(x_n;\omega))}\Rightarrow e^{-\frac{1}{2}\chi^2}$$

- the likelihood $L(\omega)\sim e^{-\frac{1}{2}\chi^2(\omega)}$ → not Gaussian (except linear model)

Regression

$$N(\mu,C)(y)=\frac{1}{\sqrt{\det(2\pi C)}}e^{-\frac{1}{2}(y-\mu)C^{-1}(y-\mu)}$$

task of regression – theoretical approach

- expanding near the maximum likelihood → Gaussian with

$$\chi^2=\chi_0^2+(\omega-\omega_0)R^{-1}(\omega-\omega_0)+\dots$$

- Gaussian confidence levels
  - the confidence surfaces are ellipsoids:
  - confidence levels in **one dimension** $\chi^2-\chi_0^2=n$

sigma-level	1	2	3	4	5
confidence	68.27%	99.54%	99.73%	99.9937%	99.99994%

Regression

$$N(\mu,C)(y)=\frac{1}{\sqrt{\det(2\pi C)}}e^{-\frac{1}{2}(y-\mu)C^{-1}(y-\mu)}$$

task of regression – theoretical approach

- Gaussian confidence levels and chi-squared distribution

$$p_d(\chi^2)=\frac{1}{2^{d/2}\Gamma(d/2)}(\chi^2)^{\frac{d}{2}-1}e^{-\frac{1}{2}\chi^2}d\chi^2$$

- probability to be inside $\chi^2<r^2$ with the incomplete Gamma function

$$P(\chi^2<r^2)=\int_0^{r^2}d\chi^2\,p_d(\chi^2)=\frac{\gamma(d/2,r^2/2)}{\Gamma(d/2)}$$

$$\gamma(a,x)=\int_0^x x^{a-1}e^{-x}dx$$
$$\Gamma(a)=\gamma(a,\infty)$$

- in Python: `scipy.special.gammaincc`

$$\frac{1}{\sqrt{\det(2\pi C)}}e^{-\frac{1}{2}(y-\mu)^T C^{-1}(y-\mu)}d^d y=\frac{1}{(2\pi)^{d/2}}e^{-\frac{1}{2}z^T z}d^d z=\frac{1}{2^{d/2}\Gamma(d/2)}(\chi^2)^{\frac{d}{2}-1}e^{-\frac{1}{2}\chi^2}d\chi^2$$

Regression

task of regression – theoretical approach

$$N(\mu,C)(y)=\frac{1}{\sqrt{\det(2\pi C)}}e^{-\frac{1}{2}(y-\mu)C^{-1}(y-\mu)}$$

- calculation with Gaussian distribution: practical rules
  - assume *y* is Gaussian
  - introduce zero-mean variables $\xi=y-\mu$
  - expected values: $\mathrm{E}(\xi)=0, \quad E(\xi_i\xi_j)=C_{ij} \quad \Rightarrow \quad E(\xi\xi)=C$
  - higher expected values are product of pairs!
    e.g.: $$E(\xi_i\xi_j\xi_k\xi_l)=C_{ij}C_{kl}+C_{ik}C_{jl}+C_{il}C_{jk}$$

Regression

task of regression – theoretical approach

$$N(\mu,C)(y)=\frac{1}{\sqrt{\det(2\pi C)}}e^{-\frac{1}{2}(y-\mu)C^{-1}(y-\mu)}$$

- hypothesis checking with chi-squared distribution:
  - assume Gaussian hypothesis and the data model are correct
  - treat the labels as independent random variables $y_n-f(x_n,\omega)\Rightarrow\xi_n$
  - compute expected value of chi-squared

$$\mathrm{E}(\chi^2)=\mathrm{E}\left(\sum_{n=1}^{N}\xi_n C_n^{-1}\xi_n\right)=\sum_{n=1}^{N}\sum_{i,j=1}^{D}(C_n^{-1})_{ij}\mathrm{E}(\xi_{ni}\xi_{nj})=N$$

- *chi-squared value over degrees of freedom is one*

Regression
Linear regression

Linear regression

simplest relation: linear function

- data are $(X,Y)=\{(x_n,y_n)\,|\,x_n\in\mathbb{R}^{N_x},\ y_n\in\mathbb{R}^{N_y},\ n=1,2,\ldots N\}$
- assume linear relation: $f(x;a,b)=ax+b$
- coefficient *a* is an matrix, *b* is a vector $[a]\Rightarrow N_x\times N_y,\ [b]\Rightarrow N_y$
- task to find the value (distribution or likelihood) of the coefficients
- simplest case, in lot of cases very powerful

Linear regression

one-dimensional case

- simplest: 1D linear regression, diagonal correlation matrix
- problem: data and labels are real numbers

$$(X,Y)=\{(x_n,y_n)\,|\,x_n,y_n\in\mathbb{R},\ n=1,2,\ldots N\}$$

- assume there is a linear relation between the two: $f(x;a,b)=ax+b$
- chi-squared function $\chi^2=\sum_{n=1}^{N}\frac{(ax_n+b-y_n)^2}{\sigma_n^2}$
- likelihood is Gaussian in a,b: $L(\omega)\sim e^{-\frac{1}{2}\chi^2(\omega)}=e^{\sum_{n=1}^{N}\frac{(ax_n+b-y_n)^2}{2\sigma_n^2}}$

Linear regression

1D linear case

- computation: introduce new notations
  - sum of inverse variances: $\sigma^2 = \frac{1}{N} \sum_{n=1}^{N} \frac{1}{\sigma_n^2}$
  - averaging: $\langle u \rangle = \frac{\sum_n u_n / \sigma_n^2}{\sum_n 1/\sigma_n^2} \Rightarrow \frac{1}{N} \sum_{n=1}^{N} u_n$
  - correlation: $C_{uv} = \langle uv \rangle - \langle u \rangle \langle v \rangle = \langle (u - \langle u \rangle)(v - \langle v \rangle) \rangle$
  - $\bar{b} = b + a \langle x \rangle - \langle y \rangle \Rightarrow y = \langle y \rangle + a(x - \langle x \rangle) + \bar{b}$

Linear regression

1D linear case

- computation: $\chi^2=\left(a-C_{yx}C_{xx}^{-1}\right)\frac{NC_{xx}}{\sigma^2}\left(a-C_{yx}C_{xx}^{-1}\right)^T+\frac{N\overline{b^2}}{N}+\text{constant}$
  - a and $\bar{b}$ are independent Gaussian variables

$$\mu_a=C_{yx}C_{xx}^{-1},\quad C_a^{-1}=\frac{N\,C_{xx}}{\sigma^2},\quad C_{\bar{b}}^{-1}=\frac{N}{\sigma^2}$$

  - the distribution of the variable y=ax+b (with fixed x) is also Gaussian

$$\mu_y=\langle y\rangle+\mu_a\left(x-\langle x\rangle\right),\quad C_y=\left(x-\langle x\rangle\right)C_a\left(x-\langle x\rangle\right)+C_{\bar{b}}$$

Linear regression

1D linear case

- interpretation:
  - best slope value and variance

$$a = C_{yx} C_{xx}^{-1}, \quad \sigma_a^2 = \frac{1}{C_{xx} \sum\limits_n 1/\sigma_n^2} \Rightarrow \frac{\sigma^2}{\sum_n \left(x_n - \bar{x}\right)}$$

  - chi-squared expected value

$$\frac{1}{N} \sum_n \frac{\left(y_n - a x_n - b\right)^2}{\sigma_n^2} = 0.81$$

Linear regression

higher dimensional linear case

- data are $(X,Y)=\{(x_n,y_n)\,|\,x_n\in\mathbb{R}^{N_x},\ y_n\in\mathbb{R}^{N_y},\ n=1,2,\ldots N\}$
- assume linear relation: $f(x;a,b)=ax+b$
- coefficient *a* is an matrix, *b* is a vector $[a]\Rightarrow N_x\times N_y,\ \ [b]\Rightarrow N_y$
- calculation and formulae remain the same

$$a=C_{yx}C_{xx}^{-1},\qquad C_a^{-1}=\frac{N}{\sigma^2}C_{xx}$$

Linear regression

higher dimensional linear case

- example: $N_x = N_y = 2, N_{data} = 100,$

$$a_{true} = \begin{pmatrix} 1 & 2 \\ 3 & 4 \end{pmatrix}, \quad b_{true} = \begin{pmatrix} 1 \\ -1 \end{pmatrix}$$

- then the result is

$$a_{data} = \begin{pmatrix} 0.99 & 2.03 \\ 3.06 & 3.96 \end{pmatrix}, \quad b_{data} = \begin{pmatrix} 0.98 \\ -0.98 \end{pmatrix}$$

```
# linear regression in 2D
Ndata = 100

a = np.array([[1, 2],[3, 4]])
b = np.array([1,-1])
x = np.random.uniform(0,1, size=(2, Ndata))
noise = np.random.normal(0,0.1, size=(2, Ndata))
y = a@x + noise + b[:,None]

barx = np.mean(x, axis=1)
bary = np.mean(y, axis=1)
Cxx = (x-barx[:,None])@(x-barx[:,None]).T/2
Cyx = (y-bary[:,None])@(x-barx[:,None]).T/2

a1 = Cyx@np.linalg.inv(Cxx)
b1 = bary - a1@barx
omega1, b1
```

role of the individual variances

- effect of different variances → weight importance of points

- variance
  - measurement (tedious)
  - theoretical considerations (e.g. relative error is $\sigma \sim \sqrt{N}$)
  - user expectations (e.g. some points are important)

Linear regression

role of the individual variances

- correlation matrix from measurements:
  - assume that the data are i.i.d. normal $p(x) \sim \prod_{n=1}^{N} e^{-\frac{1}{2}(x_n-\mu)C^{-1}(x_n-\mu)}$
  - distribution of $\overline{x} = \frac{1}{N}\sum_{n=1}^{N} x_n$ is normal with $\mathrm{E}(\overline{x}) = \mu$ → estimate for mean
  - mean of $\overline{C_{ij}} = \frac{1}{N-1}\sum_{n=1}^{N} (x_n - \overline{x})_i (x_n - \overline{x})_j$ → $\mathrm{E}(\overline{C_{ij}}) = C_{ij}$ → variance

Linear regression

fitting in a functional basis

- in lot of applications the task is to fit a function in a basis
- one-dimensional case: $f(t)$ function, $g_\alpha(t)$ basis, and we seek a_α:

$$f(t)=\sum_{\alpha=1}^{N_{basis}} a_\alpha g_\alpha(t)$$

- examples:
  - polynomial expansion: $g_\alpha(t)=t^\alpha$ $\rightarrow$ linear fit uses 2D basis $g=(1,t)$
  - Fourier expansion: $g_\alpha(t)=e^{i\alpha t}$
  - Laplace transformation: $g_\alpha(t)=e^{-\alpha t}$

Linear regression

fitting in a functional basis

- the basis elements should be linearly independent, i.e.

$$0=\sum_{\alpha=1}^{N_{basis}} a_\alpha g_\alpha(t) \quad \Leftrightarrow \quad a_\alpha=0$$

- in practice we have data pairs $D=\{(t_n, y_n) \mid n=1,2,\dots,N\}$
- solve $y_n \approx \sum_{\alpha=1}^{N_{basis}} a_\alpha g_\alpha(t_n)$, which means minimize $\chi^2=\sum_{n=1}^{N_{data}} \frac{\left(y_n-\sum_\alpha a_\alpha g_\alpha(t_n)\right)^2}{\sigma_n^2}$
- once we get the coefficients, we can interpolate: $f(t)\approx\sum_{\alpha=1}^{N_{basis}} a_\alpha g_\alpha(t)$ *(prediction)*

Linear regression

fitting in a functional basis

- computation *(more elegant than the linear case …)*
  - introduce vector notation

$$X \in \mathbb{R}^{N \cdot N_{basis}}, \quad X_{n\alpha} = g_\alpha(t_n), \quad Y \in \mathbb{R}^N, \quad c \in \mathbb{R}^{N_{basis}} \quad \Rightarrow \quad \sum_\alpha a_\alpha g_\alpha(t_n) = X \cdot a$$

$$C \in \mathbb{R}^{N^2}, \; C = \text{diagonal}(\sigma_1, \sigma_2, \ldots, \sigma_N)$$

$$\chi^2 = \sum_{n=1}^{N_{data}} \frac{\left(Y_n - \sum_\alpha a_\alpha X_{n\alpha}\right)^2}{\sigma_n^2} = (X \cdot a - Y)^T C^{-1} (X \cdot a - Y)$$

Linear regression

fitting in a functional basis

- computation *(more elegant than the linear case …)*
  - extremum: $\frac{\partial \chi^2}{\partial a} = 0 = X^T C^{-1} (X \cdot a - Y)$
  - solution: $a = (X^T C^{-1} X)^{-1} X^T C^{-1} Y$ *(→ c.f. linear case)*
  - for uniform correlation matrix: $a = (X^T X)^{-1} X^T Y$

Linear regression

fitting in a functional basis

- example:
  - create a random function, e.g. damped oscillator with noise

$$x_{n+1}=K\left(2x_n-x_{n-1}+\sigma\xi_n\right)$$

  - choose functional basis, e.g. sin-cos

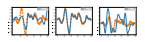
$$g_\alpha(t)=\left(\sin(\alpha t),\cos(\alpha t)\right)$$

  - use the fit formulae

Linear regression

fitting in a functional basis

- example:
  - result depends on the number of basis elements → non-monotonic!



Linear regression

fitting in a functional basis

- example:
  - result depends on the number of basis elements → non-monotonic!

Linear regression

fitting in a functional basis

- what happened?
  - we have to invert a matrix: $a=(X^T X)^{-1} X^T Y$
  - what if there are small eigenvalues?

Linear regression

eigenvalues and eigenvectors in nutshell

- eigenvectors are stabile directions of a matrix

$$M v_n = \lambda_n v_n \quad V_{in} = (v_n)_i \quad \Rightarrow \quad MV = V\Lambda, \quad \Lambda = \mathrm{diagonal}(\lambda_1, \lambda_2, \ldots, \lambda_{N_{basis}})$$

- eigenvectors form a complete basis *(in most numerical cases)*

$$M = V \Lambda V^{-1} \quad \Rightarrow \quad M^{-1} = V \Lambda^{-1} V^{-1}$$

- matrix functions, defined with Laurent series

$$f(M) = \sum_k c_k M^k = \sum_k c_k (V \Lambda V^{-1})^k = V \left(\sum_k c_k \Lambda^k \right) V^{-1} = V f(\Lambda) V^{-1}$$

Linear regression

eigenvalues and eigenvectors in nutshell – special cases

- hermitian matrices: $M = M^{+} = M^{*T}$ $\rightarrow$ real symmetric matrices
  - eigenvalues are real
  - eigenvectors are orthogonal $V^{+} V = V^{+} V = 1, \quad M = V \Lambda V^{+}$
- $M = X^{+} X$ matrices:
  - hermitian
  - eigenvalues are non-negative (positive definite)

$$M v_n = \lambda_n v_n \quad \Rightarrow \quad v_n^{+} M = \lambda_n^{*} v_n^{+} \quad \Rightarrow \quad v_n^{+} M v_m = \lambda_n^{*} v_n^{+} M v_m = \lambda_m v_n^{+} M v_m$$

$$X^{+} X v_n = \lambda_n v_n \quad \Rightarrow \quad \lambda_n = v_n^{+} X^{+} X v_n = |X v_n|^2$$

Linear regression

fitting in a functional basis

- what happened?
  - we have to invert a matrix: $a=(X^T X)^{-1} X^T Y$
  - what if there are small eigenvalues?
- a linear equation $Mx=y \;\Rightarrow\; x=M^{-1}y \;\rightarrow$ how reliable is it?
- add an eigenvector to the solution! $M(x+z v_0)=y+\lambda_0 z v_0$
- if numerical tolerance is $\epsilon \;\rightarrow\; |y+\lambda_0 z v_0| \approx |y|+\lambda_0 z<(1+\epsilon)|y| \;\Rightarrow\; z<\frac{\epsilon}{\lambda}|y|$
- if $\lambda \ll \epsilon \;\Rightarrow\; z$ can be very large!

Linear regression

fitting in a functional basis

- typical value: $\epsilon \approx 10^{-12} - 10^{-17}$
  - measurement errors
  - numerical calculation errors
- ***ill-conditioned matrices:*** there are eigenvalues $\lambda \ll 10^{-15} \lambda_{max}$ the inverse is not reliable!
- very common in higher dimensions! → in ML or AI
- never use the naive linear equation solving formulae!

Linear regression

fitting in a functional basis

- in our example: for $N_{basis} \approx 21$ we reached Python’s precision $\epsilon \sim 10^{-17}$
- after that value the fitting precision is drastically reduced

Linear regression

fitting in a functional basis

- what can we do with an ill-conditioned matrix?
  - pseudo-inverse
  - regularization

Linear regression

pseudo-inverse

- assume we can add the smallest eigenvector(s) with arbitrary coefficients
- choose that value that leads to the smallest length

$$x=\sum_n \alpha_n v_n \quad \Rightarrow \quad |x+c\,v_1|^2=(\alpha_1+c)^2+\sum_{n=2}^{N_{basis}} \alpha_n^2=\text{minimal} \quad \Rightarrow \quad c=-\alpha_1$$

- lesson: we should leave out all small eigenvectors!
- in formula: $M_{ij}^{-1}=\sum_{n=1}^{N_{basis}} \frac{1}{\lambda_n} v_{ni} v_{nj} \Rightarrow \sum_{\lambda_n>\epsilon\lambda_{max}}^{N_{basis}} \frac{1}{\lambda_n} v_{ni} v_{nj}$

Linear regression

pseudo-inverse

- in Python code

```
# pseudoinverse fitting
nus = np.linspace(1,30,Nnu)
base1 = [[np.cos(np.pi*nu*n/Ndata) for n in range(Ndata)] for nu in nus]
base2 = [[np.sin(np.pi*nu*n/Ndata) for n in range(Ndata)] for nu in nus]
f = np.array(base1+base2).T

c = np.linalg.pinv(f.T@f)@f.T@y
plt.plot(f@c)
```

Linear regression

- pseudo-inverse works well, but closely connected to linear algebra
- more general method: **regularization**
- idea: add extra term to the loss function
  - stabilize result
  - spoils close connection to data
  - tradeoff: should use in a balanced manner
- most frequently used: L1,L2 regularization, MEM

Linear regression

- L2 regularized loss

$$L=\sum_{n=1}^{N_{data}} \frac{\left(y_n - \sum_\alpha a_\alpha x_{n\alpha}\right)^2}{\sigma_n^2} + \lambda \sum_{\alpha=1}^{N_{basis}} a_\alpha^2 \;\Rightarrow\; (X \cdot a - Y)^T C^{-1} (X \cdot a - Y) + \lambda a^T a$$

$$\frac{\partial L}{\partial a} = 0 = X^T C^{-1} (X \cdot a - Y) + \lambda a \;\Rightarrow\; a = (X^T C^{-1} X + \lambda)^{-1} X^T C^{-1} Y$$

- formally: add a small number to the matrix to be inverted
- works only for positive definite matrices (true in our case)

Linear regression

- L2 regularized loss results

Linear regression

- L1 regularized loss – also for positive definite matrices

$$L=\sum_{n=1}^{N_{data}}\frac{\left(y_n-\sum_\alpha a_\alpha x_{n\alpha}\right)^2}{\sigma_n^2}+\lambda\sum_{\alpha=1}^{N_{basis}}|a_\alpha|$$

- minimization with numerical methods

Linear regression

- L1 regularization benefit: likes small coefficients
- reason: $x^2+y^2=1 \;\Rightarrow\; \min(|x|+|y|) \;\Rightarrow\; x=0, y=1$ or equivalent
- can be used to eliminate features → c.f. previous plot

Linear regression

- interpolation vs. extrapolation
- after function fitting we can predict values
- data lay in $t_n \in T_{range} = [t_{min}, t_{max}]$
  - **interpolation**: prediction for $t \in T_{range}$
  - **extrapolation**: prediction for $t \notin T_{range}$
- dangers of interpolation → overfitting
- dangers of extrapolation → …

Linear regression

- dangers of interpolation → **overfitting**
- example: fit polynomial basis to data points
- original model $y=1-x^2+\xi$ with $N_{data}=11$
- fitted model: $N_{basis}=11$
  with and or without regularization
- result:
  - exact fit, hits all points → bad interpolation
  - worse at the edges
  - regularization helps

Linear regression

- dangers of extrapolation → **avoid extrapolation**
- fitting lower and higher order polynomials
- results:
  - inside the data range all fits are correct
  - outside all fail
  - the more robust (fewer parameters)
the better
  - regularization does not help!

Linear regression

- dangers of extrapolation → **avoid extrapolation**
- fitting lower and higher order polynomials
- results:
  - inside the data range all fits are correct
  - outside all fail
  - the more robust (fewer parameters) the better
  - regularization does not help!

☠️ proceed and perish!⚠️

Linear regression

- controlling extrapolation
  - use bounded basis → avoid polynomials!
  - require asymptotic behaviour (i.e. function form at infinity) → transforms extrapolation to interpolation
  - use the model requiring the fewest parameters (robust modelling)
  - extend data in a sensible way

Nonlinear regression

Nonlinear regression

- we can have a general data model $y=f(x;\omega)$
- minimize $\chi^2(\omega)=\sum_{n=1}^{N}(y_n-f(x_n;\omega))^T C_n^{-1}(y_n-f(x_n;\omega))$
- no direct methods → recursive approach
  - steepest descent method
  - random algorithms → Metropolis method with simulated annealing
  - challenges at higher dimensions

Steepest descent method

- iteration
  - choose a starting point ω_0
  - establish an iteration $\omega_n \Rightarrow \omega_{n+1}$
  - try to ensure that $\chi^2(\omega_{n+1}) < \chi^2(\omega_n)$
- in steepest descent method: linear approximation around ω_n

$$\chi^2(\omega_n + d\omega) - \chi^2(\omega_n) = d\omega \cdot \nabla \chi^2(\omega_n) + \dots$$

- length of a vector $|v|^2 = v \cdot v > 0$ $\rightarrow$ choose $d\omega = -r \nabla \chi^2(\omega_n)$
- in linear approximation $\chi^2(\omega_n + d\omega) - \chi^2(\omega_n) = -r \left| \nabla \chi^2(\omega_n) \right|^2 + \dots < 0$

Steepest descent method

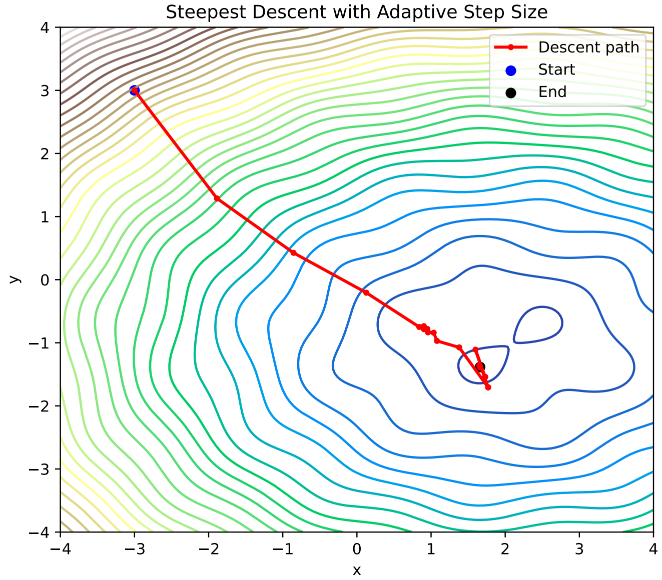
- advantages:
  - simple, low cost
  - requires only first derivative → numerically or automatic differentiation
  - flexible: stochastic and mini batch versions in high dimensions
- disadvantages:
  - slow (linear convergence), especially near minimum
  - for ill-conditioned functions long iteration path → conjugate gradient
  - sensitive to local minima → use momentum (Nesterov acceleration)
  - can lead to runaway solutions → adaptive step size, gradient clipping

Adaptive step length

- in the iteration change the step length r
  - if $\chi^2(\omega_n - r|\nabla\chi^2|) < \chi^2(\omega_n)$ $\rightarrow$ choose $r \Rightarrow \alpha_{up} r,\ \alpha_{up} > 1$
  - else choose $r \Rightarrow \alpha_{down} r,\ \alpha_{down} < 1$
  - if $r < r_{min}$ or $n_{step} > N_{iter}$ stop iteration
- 2D example with

$$f(x,y) = (x-2)^2 + (y+1)^2 + \sin 3x \sin 3y$$

$$\alpha_{up} = 1.1,\ \alpha_{down} = 0.5,\ N_{iter} = 200$$



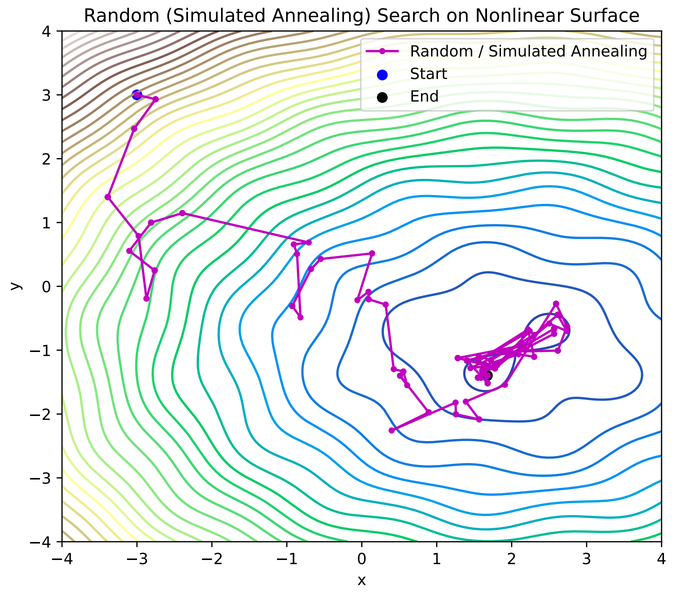
Simulated annealing

- choose a random direction n and a positive number α
- if $\chi^2(\omega_n+\alpha n)<\chi^2(\omega_n)$ $\rightarrow$ choose $\omega_{n+1}=\omega_n+\alpha n$
- else still accept $\omega_{n+1}=\omega_n+\alpha n$ with a probability $P\sim e^{-\Delta\chi^2/T}$
- T is the “temperature” controlling exploration
- the values in $\{\omega_0,\omega_1,\ldots,\omega_n,\ldots\}$ have a distribution $p(\omega)\sim e^{-\chi^2(\omega)/T}$
  - can be used in Monte Carlo simulation (Metropolis algorithm)
- if we need the minimum, decrease T $\rightarrow$ simulated annealing (SA)

$$T \Rightarrow \alpha_{cooling} T$$

Simulated annealing

- the same example as before $\alpha_{cooling} = 0.98$
- advantages:
  - no gradient is needed
  - can escape local minima
  - parallelizable
- drawbacks:
  - slow convergence
  - difficult to use in batch methods → rarely used in DNNs



Feature optimization
Perceptron
Support Vector Machine
Extreme Learning Machine

Linear Features

- we can tailor features instead of the original ones
- simplest: linear combination → looks for a hyperplane to separate classes
- implementation:
  - perceptron
  - Support Vector Machine

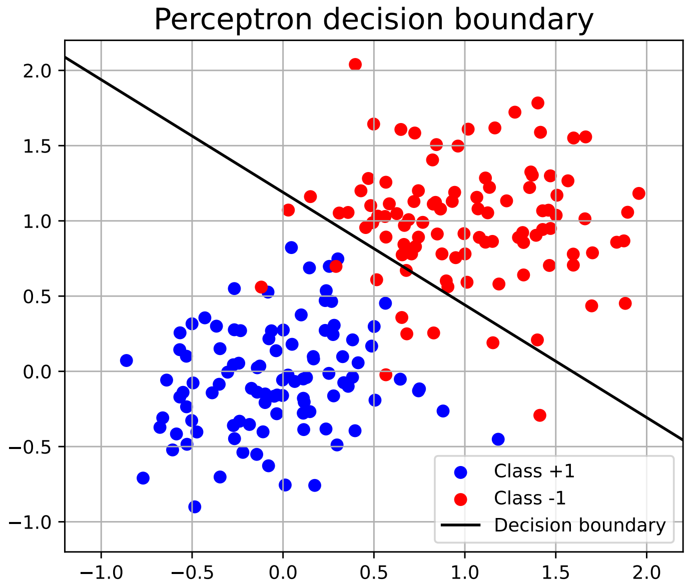
Perceptron

- one of the first AI algorithm (Rosenblatt, 1958)
  → newspapers: *“the electronic brain that can walk, talk, see, write, reproduce itself, and be conscious of its existence”*
- idea:
  - two classes with labels $y=\pm 1$, data are d dimensional $x \in \mathbb{R}^d$
  - look for a vector w and a constant b that satisfies $y = \mathrm{sign}(x \cdot w + b)$
  - possible loss function to be minimized $L = \sum_{n=1}^{N} y_n (x_n \cdot w + b)$
  - Rosenblatt’s algorithm: if we find a misclassified point x_n

$$w \Rightarrow w + \eta\, y_n x_n, \quad b \Rightarrow b + \eta\, y_n, \quad \eta > 0$$

Perceptron

- example: 2D points
- accuracy: 95.5%



Support Vector Machine

- improved perceptron to maximize margin
- logic:
  - define a *w* vector → the selective coordinate is $x \cdot w$
  - try to separate the two sets using this feature
  - for optimal w use 1D methods (e.g. hinge loss)
  - in the training use the nearest point to write up w → support vectors
  - Python package: svm

Support Vector Machine

```
# Support Vector Machine example
from sklearn import svm
from sklearn.metrics import accuracy_score

clf = svm.SVC(kernel='linear', C=1.0)
clf.fit(X, y)

# 🔮 Predict on the test set
y_pred = clf.predict(X_test)

# 🧮 Evaluate
acc = accuracy_score(y_test, y_pred)

→ acc = 96.5%
```

Support Vector Machine

- we can use also nonlinear combinations (kernel)
- most used: Gaussian kernel
  - around each point we take a “bump” $x \rightarrow e^{-\gamma(x-x_0)^2}$
  - complete kernel is a linear combination (using ±1 as labels) $$x \rightarrow \sum_n \alpha_n y_n e^{-\gamma(x-x_n)^2} + b$$
  - resulting surface is a landscape with hills and valleys
  - in practice it is enough to consider only a small number of points (support vectors) to represent the surface

Support Vector Machine

- we can use arbitrary kernel $x \rightarrow \sum_n \alpha_n y_n K(x_i, x; \Theta) + b$
- fitting and learning:
  - linear combination parameters α_n, b are learned using data
  - kernel parameters Θ are fixed in the learning process, and tuned for best performance (using grid search, cross validation)
  - Deep Neural Networks work in a similar manner, there the functional space is a nested (layered) function composition.

$$x \rightarrow W_n \sigma(W_{n-1} \sigma(\ldots \sigma(W_1(x))\ldots))$$

  - decision based on a single (although) complicated feature

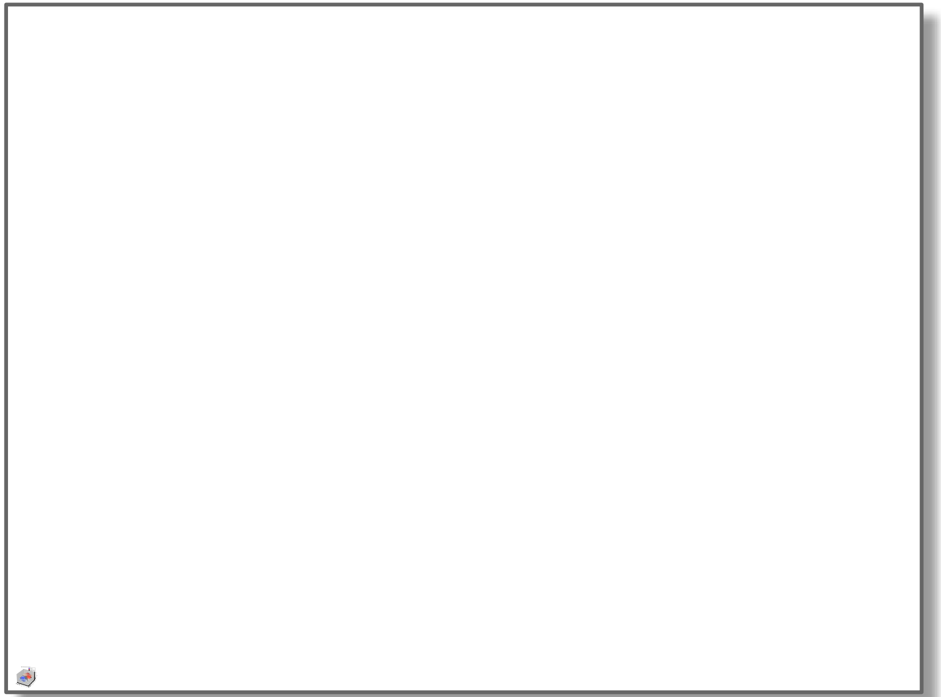
Support Vector Machine

- in the code we should use the *‘rbf’* kernel *(Radial Basis Function)*

```
# Random Forest example
from sklearn import svm
from sklearn.metrics import accuracy_score

clf = svm.SVC(kernel='linear', C=1.0)
clf.fit(X, y)

# 🔮 Predict on the test set
y_pred = clf.predict(X_test)

# 🧮 Evaluate
acc = accuracy_score(y_test, y_pred)

→ acc = 97.5%
```

Support Vector Machine

- in the code we should use the *‘rbf’* kernel *(Radial Basis Function)*



Linear representation of data distribution

Gaussian Mixture Models

- interconnected data are often sit in a compact region → ellipsoid
- find the ellipsoid containing most of the data! $p(x) \sim \prod_{n=1}^{N} e^{-\frac{1}{2}(x_n-\mu)C^{-1}(x_n-\mu)}$

$$L(\mu, C) = p(x \mid \mu, C) \ \text{ maximum } \ \Rightarrow \mu \approx \frac{1}{N}\sum_{n=1}^{N} x_n, \ C_{ij} \approx \frac{1}{N-1}\sum_{n=1}^{N}(x_n-\bar{x})_i (x_n-\bar{x})_j$$

- probability level → "distance" $\chi^2 = (x_n-\mu)C^{-1}(x_n-\mu)$
- equidistance surfaces $\chi^2 = \text{constant}$ → ellipsoids
- generate points: $x = \mu + r\sum_{a=1}^{\dim} \alpha_a \sqrt{\lambda_a} v_a \ \rightarrow \ \sum_{a=1}^{\dim} \alpha_a^2 = 1, \ C v_a = \lambda_a v_a$

Gaussian Mixture Models

- technical slide: equipotential surfaces from correlation matrix
- we want to satisfy $\chi^2 = (x-\mu) C^{-1} (x-\mu) = r^2$
- eigensystem $C v_a = \lambda_a v_a$, eigenvectors orthogonal
- ansatz: $x = \mu + r \sum_{a=1}^{\dim} \alpha_a \sqrt{\lambda_a} v_a \Rightarrow x - \mu = r \sum_{a=1}^{\dim} \alpha_a \sqrt{\lambda_a} v_a$
- verify: $\chi^2 = r^2 \sum_{a,b=1}^{\dim} (\alpha_a \sqrt{\lambda_a})(\alpha_b \sqrt{\lambda_b}) v_a C^{-1} v_b = r^2 \sum_{a,b=1}^{\dim} (\alpha_a \sqrt{\lambda_a})(\alpha_b \sqrt{\lambda_b}) \lambda_a \delta_{ab} = r^2$

Principal Component Analysis (PCA)

- technical slide: generate data with given μ, C
  - data with normal distribution: ξ
  - transform it $\eta = \sqrt{C}\,\xi + \mu$
  - check:

$$\mathrm{E}(\eta) = \mu$$

$$\mathrm{E}((\eta-\mu)\otimes(\eta-\mu)) = \sqrt{C}\,\mathrm{E}(\xi\otimes\xi)\sqrt{C} = C$$

  - for 2D data we can directly specify the direction: $ev = \begin{pmatrix} \cos\phi & -\sin\phi \\ \sin\phi & \cos\phi \end{pmatrix}$

```
# generating Gaussian with given mu and C
def generate_Gaussian(mu, C, Ndata):
    dim=mu.shape[0]
    xi = np.random.normal(0,1, (dim, Ndata))
    lam,ev =np.linalg.eigh(C)
    sqrt_C = ev@np.diag(np.sqrt(lam))@ev.T
    return sqrt_C@xi + mu
```

Gaussian Mixture Models

- example: 2D points $C=\begin{pmatrix} 2.5 & 1 \\ 1 & 1 \end{pmatrix} \Rightarrow \lambda=(0.5,3),\ ev=\frac{1}{\sqrt{5}}\begin{pmatrix} 1 & -2 \\ 2 & 1 \end{pmatrix}$
  - generate points
  - numerically determine correlation matrix and eigensystem
  - draw ellipses containing 99% of data → about 3σ in 2D

Gaussian Mixture Models

- if we have several classes → Naive Bayes models
  - assume that each class are independent and points lie in an ellipsoid → can be represented by Gaussian distribution
  - determine the probability to belong to each classes separately

```
# Naive Bayes classification
from sklearn.naive_bayes import GaussianNB

nb = GaussianNB()
nb.fit(X_train, y_train)
y_pred = nb.predict(X_test)

→ accuracy: 96%
```

Gaussian Mixture Models

- unsupervised generalization → GMM
  - assume the probability distribution is a **sum** of Gaussian terms
  - fix the number of clusters → *do not provide labels (unsupervised)*
  - find the best parameters by maximizing the likelihood

$$L(\pi, \mu, C \mid x) = p(x \mid \pi, \mu, C) = \sum_{k=1}^{K} \pi_k N(x; \mu_k, C_k)$$

  - we need convex combinations $\pi_k \in [0,1], \sum_{k=1}^{K} \pi_k = 1$
  - use Expectation–Maximization (EM) algorithm

Gaussian Mixture Models

- example: clustering in 2D

```
# Gaussian Mixture Models
from sklearn.mixture import GaussianMixture

gmm = GaussianMixture(n_components=2, covariance_type='full', random_state=0, n_init=10)gmm.fit(X)
```

Gaussian Mixture Models

- technical slide: the Expectation–Maximization (EM) algorithm
  - start from an initial guess π, μ, C
  - compute “responsibilities” (closest component weight) $r_{ik} = \dfrac{\pi_k N(x_i; \mu_k, C_k)}{\sum_j \pi_j N(x_i; \mu_j, C_j)}$
  - redefine parameters (attract to closest cluster)

$$N_k = \sum_i r_{ik}, \quad \pi_k = \frac{N_k}{N}, \quad \mu_k = \frac{1}{N_k} \sum_i x_i r_{ik}, \quad C_k = \frac{1}{N_k} \sum_i (x_i - \mu_k)(x_i - \mu_k)^T$$

  - repeat until converges

Gaussian Mixture Models

- technical slide: the Expectation–Maximization (EM) algorithm

Principal Component Analysis (PCA)

- choose an orthonormal coordinate system aligned with the data!
- we fitted an ellipsoid → find the eigenvectors
  - smallest eigenvalues: data vary the least
  - largest eigenvalues: data vary the most

Principal Component Analysis (PCA)

- PCA: keep only largest eigenvalues
  - compression → descriptive features
  - noise reduction
  - representation of the largest changes
- small eigenvalues:
  - approximately constant for all data in a class
  - classification → selective features
  - laws

Principal Component Analysis (PCA)

- technical slide: compression with PCA

$$\text{data}=\{x_n \in \mathbb{R}^d \mid n=1,2,\ldots,N\} \quad X_{ni}=(x_n)_i$$

$$\mu_i=\frac{1}{N}\sum_n X_{ni}, \quad \overline{X}_{ni}=X_{ni}-\mu_i \quad C=\overline{X}^T\overline{X}$$

$$C v_a=\lambda_a v_a, \quad |v_a|=1 \quad \Rightarrow \quad \mathrm{E}\left(\left(\bar{x}\cdot v_a\right)^2\right)=\mathrm{E}\left(v_a^T\overline{X}^T\overline{X}v_a\right)=\mathrm{E}\left(v_a C v_a\right)=\lambda_a$$

- eigenvalue = projection value squared
- PCA values: $Y_{na}=v_a^T\bar{x}_n$ for the largest eigenvalues → fewer data

Principal Component Analysis (PCA)

- technical slide: data reconstruction with PCA
- most information is in the subspace of largest eigenvalues

$$\bar{x}_n = \sum_{a=1}^{d} v_a \left(v_a^T \cdot \bar{x}_n \right) \approx \sum_{\lambda_a / \lambda_{max} > r_{min}} v_a \left(v_a^T \bar{x}_n \right) = \sum_{\lambda_a / \lambda_{max} > r_{min}} Y_{na} v_a$$

- therefore $x_n = \sum_{\lambda_a / \lambda_{max} > r_{min}} Y_{na} v_a + \mu_i$